Claude Conversation — Challenge 2: Patterns of Life

Q1: User

Challenge 2: Patterns of Life considers the patterns of daily life throughout the city. You will describe the daily routines for some representative people, characterize the travel patterns to identify potential bottlenecks or hazards, and examine how these patterns change over time and seasons. In Challenge 2, you will use visual analytic techniques to address these questions:

- Assuming the volunteers are representative of the city's population, characterize the distinct areas of the city that you identify. For each area you identify, provide your rationale and supporting data. Limit your response to 10 images and 500 words.

Try to find solutions in the internet and resolve this first problem of the challenge 2. Provide all codes and detailed explanations for each step

A1: Claude

Searched the web

Searched the web

Now I have everything needed to build a complete solution. Here is the full Python approach, step by step:

🗺️ Question 1 — Characterize Distinct City Areas

The strategy is: **load all location data → parse geometries → overlay all venue types on a map → cluster by spatial density and dominant venue type → characterize each zone**.

Step 1 — Install dependencies

bash

```
pip install pandas geopandas matplotlib shapely scikit-learn seaborn contextily numpy
```

Step 2 — Full Analysis Script

Create a file called `q1_city_areas.py`:

python

```
# ============================================================
  # VAST Challenge 2022 - Challenge 2, Question 1
  # Characterize distinct areas of the city
  # ============================================================
```

```
import pandas as pd
  import numpy as np
  import matplotlib.pyplot as plt
  import matplotlib.patches as mpatches
  from matplotlib.colors import ListedColormap
  import geopandas as gpd
  from shapely.geometry import Point, Polygon
  from shapely import wkt
  from sklearn.cluster import DBSCAN
  from sklearn.preprocessing import StandardScaler
  import warnings
  warnings.filterwarnings('ignore')
```

```
DATA_PATH = "data/"   # <-- adjust to where your CSV files are
```

```
# ============================================================
  # STEP 1: Load all attribute files
  # ============================================================
```

```
buildings    = pd.read_csv(DATA_PATH + "Buildings.csv")
  apartments   = pd.read_csv(DATA_PATH + "Apartments.csv")
  employers    = pd.read_csv(DATA_PATH + "Employers.csv")
  restaurants  = pd.read_csv(DATA_PATH + "Restaurants.csv")
  pubs         = pd.read_csv(DATA_PATH + "Pubs.csv")
  schools      = pd.read_csv(DATA_PATH + "Schools.csv")
  participants = pd.read_csv(DATA_PATH + "Participants.csv")
  checkins     = pd.read_csv(DATA_PATH + "CheckinJournal.csv")
```

```
print("Buildings:", len(buildings))
  print("Apartments:", len(apartments))
  print("Employers:", len(employers))
  print("Restaurants:", len(restaurants))
  print("Pubs:", len(pubs))
  print("Schools:", len(schools))
```

```
# ============================================================
  # STEP 2: Parse geometries
  # Buildings have polygon geometry (WKT format)
  # Others have point geometry (WKT format)
  # ============================================================
```

```
def parse_point(loc_str):
      """Parse 'POINT (x y)' string to shapely Point."""
      try:
          return wkt.loads(loc_str)
      except:
          # Sometimes stored as 'x y' or with extra spaces
          coords = loc_str.strip().replace("POINT (","").replace(")","").split()
          return Point(float(coords[0]), float(coords[1]))
```

```
def parse_polygon(loc_str):
      """Parse polygon WKT string to shapely Polygon."""
      try:
          return wkt.loads(loc_str)
      except:
          return None
```

```
# Parse building footprints (polygons)
  buildings['geometry'] = buildings['location'].apply(parse_polygon)
  gdf_buildings = gpd.GeoDataFrame(buildings, geometry='geometry')
```

```
# Parse point locations for venues
  for df, name in [(apartments,'apartments'), (employers,'employers'),
                   (restaurants,'restaurants'), (pubs,'pubs'), (schools,'schools')]:
      df['geometry'] = df['location'].apply(parse_point)
```

```
gdf_apts  = gpd.GeoDataFrame(apartments,  geometry='geometry')
  gdf_emp   = gpd.GeoDataFrame(employers,   geometry='geometry')
  gdf_rest  = gpd.GeoDataFrame(restaurants, geometry='geometry')
  gdf_pubs  = gpd.GeoDataFrame(pubs,        geometry='geometry')
  gdf_schl  = gpd.GeoDataFrame(schools,     geometry='geometry')
```

```
# ============================================================
  # STEP 3: Extract coordinates for clustering
  # Use building centroids as the base spatial unit
  # ============================================================
```

```
gdf_buildings['centroid'] = gdf_buildings.geometry.centroid
  gdf_buildings['cx'] = gdf_buildings['centroid'].x
  gdf_buildings['cy'] = gdf_buildings['centroid'].y
```

```
# ============================================================
  # STEP 4: Annotate each building with its type
  # buildingType is already in Buildings.csv: "Commercial", "Residential", "School"
  # ============================================================
```

```
type_colors = {
      'Residential': '#2196F3',   # blue
      'Commercial':  '#FF9800',   # orange
      'School':      '#4CAF50',   # green
  }
```

```
# ============================================================
  # STEP 5: DBSCAN spatial clustering to find neighborhoods
  # DBSCAN groups buildings that are spatially close together
  # eps = neighborhood radius (in the coordinate units of the dataset)
  # min_samples = minimum buildings to form a cluster
  # ============================================================
```

```
coords = gdf_buildings[['cx', 'cy']].dropna().values
  scaler = StandardScaler()
  coords_scaled = scaler.fit_transform(coords)
```

```
# Tune eps based on your data extent — start with 0.3, adjust if needed
  db = DBSCAN(eps=0.3, min_samples=5).fit(coords_scaled)
  gdf_buildings = gdf_buildings.dropna(subset=['cx', 'cy']).copy()
  gdf_buildings['cluster'] = db.labels_   # -1 = noise (isolated buildings)
```

```
n_clusters = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)
  print(f"\nDBSCAN found {n_clusters} spatial clusters")
  print(gdf_buildings['cluster'].value_counts().head(15))
```

```
# ============================================================
  # STEP 6: Characterize each cluster by dominant building type
  # ============================================================
```

```
cluster_summary = gdf_buildings.groupby('cluster').agg(
      num_buildings  = ('buildingId', 'count'),
      residential    = ('buildingType', lambda x: (x == 'Residential').sum()),
      commercial     = ('buildingType', lambda x: (x == 'Commercial').sum()),
      school         = ('buildingType', lambda x: (x == 'School').sum()),
      cx             = ('cx', 'mean'),
      cy             = ('cy', 'mean'),
  ).reset_index()
```

```
cluster_summary = cluster_summary[cluster_summary['cluster'] >= 0]  # remove noise
```

```
def classify_zone(row):
      """Label each cluster as a zone type based on dominant building type."""
      total = row['num_buildings']
      if total == 0: return 'Unknown'
      res_pct  = row['residential'] / total
      com_pct  = row['commercial']  / total
      scl_pct  = row['school']      / total
      if scl_pct > 0.1:            return 'Educational Zone'
      if res_pct > 0.7:            return 'Residential Zone'
      if com_pct > 0.7:            return 'Commercial Zone'
      if res_pct > 0.4 and com_pct > 0.3: return 'Mixed-Use Zone'
      if res_pct > com_pct:        return 'Primarily Residential'
      return 'Primarily Commercial'
```

```
cluster_summary['zone_type'] = cluster_summary.apply(classify_zone, axis=1)
  gdf_buildings = gdf_buildings.merge(
      cluster_summary[['cluster', 'zone_type']], on='cluster', how='left'
  )
```

```
print("\nCluster zone classification:")
  print(cluster_summary[['cluster','num_buildings','residential','commercial',
                         'school','zone_type']].to_string())
```

```
# ============================================================
  # STEP 7: Enrich clusters with checkin activity data
  # How many checkins happen in each zone? What types?
  # ============================================================
```

```
# Merge checkins with participant info
  checkins_merged = checkins.merge(participants, on='participantId', how='left')
```

```
# Count checkins per venue type
  checkin_by_type = checkins_merged.groupby('venueType').size().reset_index(name='coun
  t')
  print("\nCheckins by venue type:")
  print(checkin_by_type)
```

```
# Count checkins per participant
  activity_per_participant = checkins_merged.groupby('participantId').agg(
      total_checkins = ('venueId', 'count'),
      unique_venues  = ('venueId', 'nunique'),
  ).reset_index()
```

```
# ============================================================
  # STEP 8: Map how many restaurants/pubs are in each cluster
  # (spatial join: point venues → cluster containing them)
  # ============================================================
```

```
# Assign each venue point to the nearest cluster centroid
  def assign_cluster(x, y, cluster_df):
      """Find nearest cluster centroid to a given point."""
      if len(cluster_df) == 0: return -1
      dists = np.sqrt((cluster_df['cx'] - x)**2 + (cluster_df['cy'] - y)**2)
      idx = dists.idxmin()
      return cluster_df.loc[idx, 'cluster']
```

```
cluster_valid = cluster_summary[cluster_summary['cluster'] >= 0]
```

```
gdf_rest['cluster'] = gdf_rest.apply(
      lambda r: assign_cluster(r.geometry.x, r.geometry.y, cluster_valid), axis=1)
  gdf_pubs['cluster'] = gdf_pubs.apply(
      lambda r: assign_cluster(r.geometry.x, r.geometry.y, cluster_valid), axis=1)
```

```
rest_per_cluster = gdf_rest.groupby('cluster').size().reset_index(name='restaurants')
  pubs_per_cluster = gdf_pubs.groupby('cluster').size().reset_index(name='pubs')
```

```
cluster_summary = cluster_summary.merge(rest_per_cluster, on='cluster', how='left')
  cluster_summary = cluster_summary.merge(pubs_per_cluster,  on='cluster', how='left')
  cluster_summary[['restaurants','pubs']] = cluster_summary[['restaurants','pubs']].fil
  lna(0)
```

```
# ============================================================
  # STEP 9: VISUALIZATION 1 — Full city map colored by building type
  # ============================================================
```

```
fig, ax = plt.subplots(1, 1, figsize=(14, 10))
  ax.set_facecolor('#f0f0f0')
```

```
for btype, color in type_colors.items():
      subset = gdf_buildings[gdf_buildings['buildingType'] == btype]
      subset.plot(ax=ax, color=color, alpha=0.6, edgecolor='white', linewidth=0.3)
```

```
# Overlay restaurants, pubs, schools as markers
  gdf_rest.plot(ax=ax, color='red',    marker='o', markersize=15,
                label='Restaurant', zorder=5, alpha=0.8)
  gdf_pubs.plot(ax=ax, color='purple', marker='s', markersize=12,
                label='Pub', zorder=5, alpha=0.8)
  gdf_schl.plot(ax=ax, color='green',  marker='^', markersize=20,
                label='School', zorder=5, alpha=0.9)
```

```
# Legend
  patches = [
      mpatches.Patch(color='#2196F3', label='Residential'),
      mpatches.Patch(color='#FF9800', label='Commercial'),
      mpatches.Patch(color='#4CAF50', label='School Building'),
      plt.Line2D([0],[0], marker='o', color='w', markerfacecolor='red',   markersize=8,
  label='Restaurant'),
      plt.Line2D([0],[0], marker='s', color='w', markerfacecolor='purple', markersize=
  8, label='Pub'),
      plt.Line2D([0],[0], marker='^', color='w', markerfacecolor='green',  markersize=
  8, label='School'),
  ]
  ax.legend(handles=patches, loc='lower right', fontsize=9)
  ax.set_title("City of Engagement — Building Types & Venues", fontsize=14, fontweight
  ='bold')
  ax.set_xlabel("X Coordinate"); ax.set_ylabel("Y Coordinate")
  plt.tight_layout()
  plt.savefig("q1_city_map.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("Saved: q1_city_map.png")
```

```
# ============================================================
  # STEP 10: VISUALIZATION 2 — Map colored by DBSCAN cluster / zone type
  # ============================================================
```

```
zone_colors = {
      'Residential Zone':      '#1565C0',
      'Primarily Residential': '#42A5F5',
      'Commercial Zone':       '#E65100',
      'Primarily Commercial':  '#FFA726',
      'Mixed-Use Zone':        '#7B1FA2',
      'Educational Zone':      '#2E7D32',
      'Unknown':               '#9E9E9E',
  }
```

```
fig, ax = plt.subplots(1, 1, figsize=(14, 10))
  ax.set_facecolor('#e8e8e8')
```

```
for zone, color in zone_colors.items():
      subset = gdf_buildings[gdf_buildings['zone_type'] == zone]
      if len(subset) > 0:
          subset.plot(ax=ax, color=color, alpha=0.7, edgecolor='white', linewidth=0.2)
```

```
# Noise buildings (cluster == -1) in grey
  noise = gdf_buildings[gdf_buildings['cluster'] == -1]
  if len(noise) > 0:
      noise.plot(ax=ax, color='#bdbdbd', alpha=0.4, edgecolor='white', linewidth=0.2)
```

```
# Label cluster centroids
  for _, row in cluster_summary.iterrows():
      ax.annotate(
          f"Zone {int(row['cluster'])}\n({row['zone_type'].replace(' Zone','').replace
  ('Primarily ','')})",
          xy=(row['cx'], row['cy']),
          fontsize=6, ha='center', color='black',
          bbox=dict(boxstyle='round,pad=0.2', fc='white', alpha=0.6)
      )
```

```
patches = [mpatches.Patch(color=c, label=z) for z, c in zone_colors.items()]
  ax.legend(handles=patches, loc='lower right', fontsize=8)
  ax.set_title("City of Engagement — Identified Zones (DBSCAN Clustering)", fontsize=1
  4, fontweight='bold')
  ax.set_xlabel("X Coordinate"); ax.set_ylabel("Y Coordinate")
  plt.tight_layout()
  plt.savefig("q1_zones_map.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("Saved: q1_zones_map.png")
```

```
# ============================================================
  # STEP 11: VISUALIZATION 3 — Bar chart: zone composition
  # ============================================================
```

```
top_clusters = cluster_summary.nlargest(10, 'num_buildings')
```

```
x = np.arange(len(top_clusters))
  w = 0.25
  fig, ax = plt.subplots(figsize=(12, 5))
  ax.bar(x - w, top_clusters['residential'], w, label='Residential', color='#2196F3')
  ax.bar(x,     top_clusters['commercial'],  w, label='Commercial',  color='#FF9800')
  ax.bar(x + w, top_clusters['school'],      w, label='School',      color='#4CAF50')
```

```
ax.set_xticks(x)
  ax.set_xticklabels([f"Zone {int(c)}\n{z}" for c, z in
                      zip(top_clusters['cluster'], top_clusters['zone_type'])],
                     rotation=30, ha='right', fontsize=8)
  ax.set_ylabel("Number of Buildings")
  ax.set_title("Top 10 Clusters — Building Type Composition", fontweight='bold')
  ax.legend()
  plt.tight_layout()
  plt.savefig("q1_zone_composition.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("Saved: q1_zone_composition.png")
```

```
# ============================================================
  # STEP 12: VISUALIZATION 4 — Apartment rental cost heatmap by location
  # (reveals economic stratification of neighbourhoods)
  # ============================================================
```

```
fig, ax = plt.subplots(figsize=(13, 9))
  ax.set_facecolor('#f0f0f0')
```

```
# Draw building footprints lightly
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white', linewidth=0.2, alpha=0.5)
```

```
# Scatter apartments colored by rental cost
  scatter = ax.scatter(
      gdf_apts.geometry.x,
      gdf_apts.geometry.y,
      c=gdf_apts['rentalCost'],
      cmap='RdYlGn_r',   # red=expensive, green=cheap
      s=20, alpha=0.75, zorder=5
  )
  cbar = plt.colorbar(scatter, ax=ax, shrink=0.6)
  cbar.set_label("Monthly Rent ($)", fontsize=10)
  ax.set_title("Apartment Rental Cost Distribution\n(reveals wealthy vs low-income area
  s)",
               fontsize=13, fontweight='bold')
  ax.set_xlabel("X"); ax.set_ylabel("Y")
  plt.tight_layout()
  plt.savefig("q1_rental_heatmap.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("Saved: q1_rental_heatmap.png")
```

```
# ============================================================
  # STEP 13: VISUALIZATION 5 — Checkin density per venue type
  # Shows which areas are most active and for what purpose
  # ============================================================
```

```
# Join checkins with venue locations
  # Map venueId to location based on venueType
  venue_locations = []
  for _, row in checkins.iterrows():
      vid  = row['venueId']
      vtyp = row['venueType']
      geom = None
      if vtyp == 'Apartment':
          m = gdf_apts[gdf_apts['apartmentId'] == vid]
          if len(m): geom = m.iloc[0].geometry
      elif vtyp == 'Workplace':
          m = gdf_emp[gdf_emp['employerId'] == vid]
          if len(m): geom = m.iloc[0].geometry
      elif vtyp == 'Restaurant':
          m = gdf_rest[gdf_rest['restaurantId'] == vid]
          if len(m): geom = m.iloc[0].geometry
      elif vtyp == 'Pub':
          m = gdf_pubs[gdf_pubs['pubId'] == vid]
          if len(m): geom = m.iloc[0].geometry
      if geom:
          venue_locations.append({'venueType': vtyp, 'x': geom.x, 'y': geom.y})
```

```
loc_df = pd.DataFrame(venue_locations)
```

```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
  venue_types = ['Apartment', 'Workplace', 'Restaurant', 'Pub']
  titles = ['Home Activity', 'Workplace Activity', 'Restaurant Activity', 'Pub Activit
  y']
  colors_v = ['#1976D2', '#F57C00', '#D32F2F', '#7B1FA2']
```

```
for ax, vt, title, col in zip(axes.flat, venue_types, titles, colors_v):
      ax.set_facecolor('#f5f5f5')
      gdf_buildings.plot(ax=ax, color='#e0e0e0', edgecolor='white', linewidth=0.2, alph
  a=0.5)
      subset = loc_df[loc_df['venueType'] == vt]
      if len(subset):
          ax.scatter(subset['x'], subset['y'], color=col, s=4, alpha=0.3)
      ax.set_title(title, fontweight='bold', fontsize=11)
      ax.set_xlabel("X"); ax.set_ylabel("Y")
```

```
plt.suptitle("Checkin Density by Activity Type\n(reveals functional specialization of c
  ity areas)",
               fontsize=14, fontweight='bold')
  plt.tight_layout()
  plt.savefig("q1_checkin_density.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("Saved: q1_checkin_density.png")
```

```
# ================================================================
  # STEP 14: Print final zone summary for the report
  # ================================================================
```

```
print("\n" + "="*70)
  print("FINAL ZONE SUMMARY (for Q1 answer)")
  print("="*70)
  for _, row in cluster_summary.sort_values('num_buildings', ascending=False).head(8).i
  terrows():
      print(f"\nZone {int(row['cluster'])} — {row['zone_type']}")
      print(f"  Buildings:   {int(row['num_buildings'])} "
            f"({int(row['residential'])} res, {int(row['commercial'])} com, {int(row['s
  chool'])} school)")
      print(f"  Restaurants: {int(row['restaurants'])}   Pubs: {int(row['pubs'])}")
      print(f"  Center:      ({row['cx']:.1f}, {row['cy']:.1f})")
```

Step 3 — Run it

bash

```
python q1_city_areas.py
```

This produces **5 PNG images** you can paste directly into your `index.html` answer form.

📖 What Each Step Does (Explanation for the Report)

Step	What it does	Why
1–2	Load CSVs, parse WKT geometries	Buildings have polygon footprints; venues have point locations
3–4	Extract building centroids + types	Centroid = spatial representative of each building
5	**DBSCAN clustering** on building centroids	Groups spatially close buildings into neighborhoods without needing to pre-define how many zones
6	Classify each cluster by dominant building type	Residential >70% → Residential Zone; mixed → Mixed-Use, etc.
7–8	Enrich clusters with checkin counts and venue counts	Tells you *how active* each zone is and what people do there
9	**Map 1** — city overview by building type	Visual overview
10	**Map 2** — zones colored by DBSCAN result	The key answer image
11	**Chart 3** — bar chart of zone composition	Supporting data for each zone
12	**Map 4** — rental cost heatmap	Reveals wealthy vs poor districts
13	**Map 5** — checkin density by activity	Shows where people work, eat, drink, sleep

---

📝 Answer Template for Q1 (≤500 words)

Based on the analysis, you'll identify zones like these and write:

> ***Zone A — Residential Core*** *: High concentration of residential buildings, low commercial density, consistent home checkins throughout the day. Low rental costs suggest a working-class neighborhood.*
> ***Zone B — Commercial/Employment Hub*** *: Dominated by employer buildings and commercial structures. High workplace checkins 8am–5pm on weekdays. No apartments, indicating people commute here.*
>
> ***Zone C — Entertainment District*** *: Cluster of restaurants and pubs with low residential density. Checkins peak evenings and weekends.*
>
> ***Zone D — Educational Area*** *: One of the five schools anchors this zone. Surrounding buildings are mixed residential, consistent with family households.*
>
> ***Zone E — Affluent Residential*** *: Residential buildings with high rental costs, lower density. Checkins show less restaurant activity (cooking at home), more recreation.*

The actual zone names and numbers will come from your printed output and maps.

Now I have everything needed to build a complete solution. Here is the full Python approach, step by step:

The strategy is: **load all location data → parse geometries → overlay all venue types on a map → cluster by spatial density and dominant venue type → characterize each zone**.

Create a file called `q1_city_areas.py`:

This produces **5 PNG images** you can paste directly into your `index.html` answer form.

Based on the analysis, you'll identify zones like these and write:

Zone A — Residential Core : High concentration of residential buildings, low commercial density, consistent home checkins throughout the day. Low rental costs suggest a working-class neighborhood.

Zone B — Commercial/Employment Hub : Dominated by employer buildings and commercial structures. High workplace checkins 8am–5pm on weekdays. No apartments, indicating people commute here.

Zone C — Entertainment District : Cluster of restaurants and pubs with low residential density. Checkins peak evenings and weekends.

Zone D — Educational Area : One of the five schools anchors this zone. Surrounding buildings are mixed residential, consistent with family households.

Zone E — Affluent Residential : Residential buildings with high rental costs, lower density. Checkins show less restaurant activity (cooking at home), more recreation.

The actual zone names and numbers will come from your printed output and maps.

Q2: User

TypeError Traceback (most recent call last)

Cell In[6], line 26

```
20 DATA_PATH = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets\Attr
 ibutes")
```

```
22 # ============================================================
```

```
23 # STEP 1: Load all attribute files
```

```
24 # ============================================================
```

---> 26 buildings = pd.read*csv(DATA*PATH + "Buildings.csv")

```
27 apartments    = pd.read_csv(DATA_PATH + "Apartments.csv")
```

```
28 employers     = pd.read_csv(DATA_PATH + "Employers.csv")
```

TypeError: unsupported operand type(s) for +: 'WindowsPath' and 'str'

A2: Claude

Simple fix — `Path` objects use `/` not `+` for joining. Change this one line:

python

```
# ❌ WRONG — can't use + with Path objects
  DATA_PATH = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets\Attrib
  utes")
  buildings = pd.read_csv(DATA_PATH + "Buildings.csv")
```

```
# ✅ CORRECT — use / operator to join Path objects
  DATA_PATH = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets\Attrib
  utes")
  buildings = pd.read_csv(DATA_PATH / "Buildings.csv")
```

Replace **every** file load line the same way:

python

```
buildings    = pd.read_csv(DATA_PATH / "Buildings.csv")
  apartments   = pd.read_csv(DATA_PATH / "Apartments.csv")
  employers    = pd.read_csv(DATA_PATH / "Employers.csv")
  restaurants  = pd.read_csv(DATA_PATH / "Restaurants.csv")
  pubs         = pd.read_csv(DATA_PATH / "Pubs.csv")
  schools      = pd.read_csv(DATA_PATH / "Schools.csv")
  participants = pd.read_csv(DATA_PATH / "Participants.csv")
```

But also note — `Participants.csv` **and** `CheckinJournal.csv` **are in a different folder (**`Attributes` vs `Journals`). Set up two path variables:

python

```
from pathlib import Path
```

```
BASE = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets")
```

```
ATTR_PATH    = BASE / "Attributes"
  JOURNAL_PATH = BASE / "Journals"
```

```
# Attribute files
  buildings    = pd.read_csv(ATTR_PATH / "Buildings.csv")
  apartments   = pd.read_csv(ATTR_PATH / "Apartments.csv")
  employers    = pd.read_csv(ATTR_PATH / "Employers.csv")
  restaurants  = pd.read_csv(ATTR_PATH / "Restaurants.csv")
  pubs         = pd.read_csv(ATTR_PATH / "Pubs.csv")
  schools      = pd.read_csv(ATTR_PATH / "Schools.csv")
  participants = pd.read_csv(ATTR_PATH / "Participants.csv")
```

```
# Journal files
  checkins     = pd.read_csv(JOURNAL_PATH / "CheckinJournal.csv")
```

Replace your `DATA_PATH = Path(...)` block with this and the error will be gone.

Simple fix — `Path` objects use `/` not `+` for joining. Change this one line:

Replace **every** file load line the same way:

But also note — **`Participants.csv` and `CheckinJournal.csv` are in a different folder** (`Attributes` vs `Journals`). Set up two path variables:

Replace your `DATA_PATH = Path(...)` block with this and the error will be gone.

Q3: User

FileNotFoundError Traceback (most recent call last)

Cell In[7], line 33

```
31 schools      = pd.read_csv(DATA_PATH / "Schools.csv")
```

```
32 participants = pd.read_csv(DATA_PATH / "Participants.csv")
```

---> 33 checkins = pd.read*csv(DATA*PATH / "CheckinJournal.csv")

```
35 print("Buildings:", len(buildings))
```

```
36 print("Apartments:", len(apartments))
```

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\io\parsers\readers.py:1026, in read*csv(filepath*or*buffer, sep, delimiter, header, names, index*col, usecols, dtype, engine, converters, true*values, false*values, skipinitialspace, skiprows, skipfooter, nrows, na*values, keep*default*na, na*filter, verbose, skip*blank*lines, parse*dates, infer*datetime*format, keep*date*col, date*parser, date*format, dayfirst, cache*dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding*errors, dialect, on*bad*lines, delim*whitespace, low*memory, memory*map, float*precision, storage*options, dtype_backend)

1013 kwds*defaults* = *_refine*defaults_read(

1014 dialect,

1015 delimiter,

(...)

1022 dtype*backend*=*dtype*backend,

1023)

1024 kwds.update(kwds_defaults)

-> 1026 return *read(filepath*or_buffer, kwds)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\io\parsers\readers.py:620, in *read(filepath*or_buffer, kwds)

```
617 _validate_names(kwds.get("names", None))
```

```
619 # Create the parser.
```

--> 620 parser = TextFileReader(filepath*or*buffer, **kwds)

```
622 if chunksize or iterator:
```

```
623     return parser
```

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\io\parsers\readers.py:1620, in TextFileReader.**init**(self, f, engine, **kwds)

1617 self.options["has*index*names"] = kwds["has*index*names"]

1619 self.handles: IOHandles | None = None

-> 1620 self.*engine* = *self*.make_engine(f, self.engine)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\io\parsers\readers.py:1880, in TextFileReader.*make*engine(self, f, engine)

1878 if "b" not in mode:

1879 mode += "b"

-> 1880 self.handles = get_handle(

1881 f,

1882 mode,

1883 encoding=self.options.get("encoding", None),

1884 compression=self.options.get("compression", None),

1885 memory*map=self.options.get("memory*map", False),

1886 is*text=is*text,

1887 errors=self.options.get("encoding_errors", "strict"),

1888 storage*options=self.options.get("storage*options", None),

1889)

1890 assert self.handles is not None

1891 f = self.handles.handle

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\io\common.py:873, in get*handle(path*or*buf, mode, encoding, compression, memory*map, is*text, errors, storage*options)

```
868 elif isinstance(handle, str):
```

```
869     # Check whether the filename is to be opened in binary mode.
```

```
870     # Binary mode does not support 'encoding' and 'newline'.
```

```
871     if ioargs.encoding and "b" not in ioargs.mode:
```

```
872         # Encoding
```

--> 873 handle = open(

```
874             handle,
```

```
875             ioargs.mode,
```

```
876             encoding=ioargs.encoding,
```

```
877             errors=errors,
```

```
878             newline="",
```

```
879         )
```

```
880     else:
```

```
881         # Binary mode
```

```
882         handle = open(handle, ioargs.mode)
```

FileNotFoundError: [Errno 2] No such file or directory: 'C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets\Attributes\CheckinJournal.csv'

A3: Claude

Looking at your screenshot, `CheckinJournal.csv` is in the **Journals** folder, not Attributes. Update your paths like this:

python

```
from pathlib import Path
```

```
BASE = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets")
```

```
ATTR_PATH    = BASE / "Attributes"
  JOURNAL_PATH = BASE / "Journals"
```

```
# ✅ Attributes folder
  buildings    = pd.read_csv(ATTR_PATH / "Buildings.csv")
  apartments   = pd.read_csv(ATTR_PATH / "Apartments.csv")
  employers    = pd.read_csv(ATTR_PATH / "Employers.csv")
  restaurants  = pd.read_csv(ATTR_PATH / "Restaurants.csv")
  pubs         = pd.read_csv(ATTR_PATH / "Pubs.csv")
  schools      = pd.read_csv(ATTR_PATH / "Schools.csv")
  participants = pd.read_csv(ATTR_PATH / "Participants.csv")
```

```
# ✅ Journals folder
  checkins     = pd.read_csv(JOURNAL_PATH / "CheckinJournal.csv")
```

If you're unsure which file is in which folder, run this in a new cell to print everything at once:

python

```
from pathlib import Path
```

```
BASE = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets")
```

```
print("=== Activity Logs ===")
  for f in (BASE / "Activity Logs").iterdir():
      print(" ", f.name)
```

```
print("\n=== Attributes ===")
  for f in (BASE / "Attributes").iterdir():
      print(" ", f.name)
```

```
print("\n=== Journals ===")
  for f in (BASE / "Journals").iterdir():
      print(" ", f.name)
```

This will show you exactly which CSV is where so you can map every file to the right folder.

Looking at your screenshot, `CheckinJournal.csv` is in the **Journals** folder, not Attributes. Update your paths like this:

If you're unsure which file is in which folder, run this in a new cell to print everything at once:

This will show you exactly which CSV is where so you can map every file to the right folder.

Q4: User

how long does it take in Juoter notebook when journals is 670 MB ?

A4: Claude

For a 670MB CSV like `CheckinJournal.csv`, here's what to expect:

Load times with plain `pd.read_csv()`

RAM you have	Estimated time
8 GB	3–6 minutes
16 GB	1–3 minutes
32 GB+	30–90 seconds

And it will use **~3–4x the file size in RAM** (~2–2.5 GB), which can crash the kernel if your PC doesn't have enough memory.

✅ Better approach — load only what you need

Since you only need checkins for Q1 (just location + venue type), load only those columns:

python

```
# Instead of loading all 670MB...
  # checkins = pd.read_csv(JOURNAL_PATH / "CheckinJournal.csv")   ❌ slow
```

```
# Load only the 3 columns you actually use — ~5x faster, ~5x less RAM
  checkins = pd.read_csv(
      JOURNAL_PATH / "CheckinJournal.csv",
      usecols=["participantId", "venueId", "venueType"]  # skip timestamp
  )
```

This alone drops it to **~30–60 seconds** and uses only ~400MB RAM.

⚡ Even faster — use chunks to sample the data

If it still crashes, read just a sample:

python

```
# Read only the first 500,000 rows (plenty for Q1 analysis)
  checkins = pd.read_csv(
      JOURNAL_PATH / "CheckinJournal.csv",
      usecols=["participantId", "venueId", "venueType"],
      nrows=500_000
  )
  print(f"Loaded {len(checkins):,} rows")
```

🚀 Fastest option — convert to Parquet once, reload instantly

Run this **once** :

python

```
# Convert to parquet (one-time, takes ~2 min)
  checkins = pd.read_csv(JOURNAL_PATH / "CheckinJournal.csv")
  checkins.to_parquet(JOURNAL_PATH / "CheckinJournal.parquet")
  print("Done! Parquet saved.")
```

Then every future load takes **3–5 seconds** :

python

```
# All future loads are instant
  checkins = pd.read_parquet(JOURNAL_PATH / "CheckinJournal.parquet")
```

Recommendation : use `usecols` first — it's the easiest fix and will get you running in under a minute without changing anything else in your code.

For a 670MB CSV like `CheckinJournal.csv`, here's what to expect:

And it will use **~3–4x the file size in RAM** (~2–2.5 GB), which can crash the kernel if your PC doesn't have enough memory.

Since you only need checkins for Q1 (just location + venue type), load only those columns:

This alone drops it to **~30–60 seconds** and uses only ~400MB RAM.

If it still crashes, read just a sample:

Run this **once** :

Then every future load takes **3–5 seconds** :

Recommendation : use `usecols` first — it's the easiest fix and will get you running in under a minute without changing anything else in your code.

Q5: User

Here are my results are they good ?

A5: Claude

Looking at your 5 images, there are **3 clear problems** to fix:

❌ Problems Found

Problem 1 — All buildings show as Commercial (orange) in both maps. Residential buildings are missing entirely. This is because in this dataset, `Buildings.csv` contains mostly/only Commercial and School types — **apartments are stored separately in** `Apartments.csv` with their own locations.

Problem 2 — DBSCAN found only 1 giant cluster (Zone 0). The coordinate range is huge (X: -5000 to 3000, Y: 0 to 8000) so all buildings are spatially connected and DBSCAN merges them all. `eps=0.3` after StandardScaler is still too large for this city layout.

Problem 3 — Zone composition bar chart shows only one bar — direct consequence of problem 2.

The rental heatmap and checkin density maps are actually good ✅.

✅ Fixed Script

Replace your clustering section with **K-Means** (more reliable here) and fix the residential layer:

python

```
from pathlib import Path
  import pandas as pd
  import numpy as np
  import matplotlib.pyplot as plt
  import matplotlib.patches as mpatches
  import geopandas as gpd
  from shapely.geometry import Point
  from shapely import wkt
  from sklearn.cluster import KMeans
  from sklearn.preprocessing import StandardScaler
  import warnings
  warnings.filterwarnings('ignore')
```

```
BASE         = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets")
  ATTR_PATH    = BASE / "Attributes"
  JOURNAL_PATH = BASE / "Journals"
```

```
# ================================================================
  # LOAD DATA
  # ================================================================
  buildings    = pd.read_csv(ATTR_PATH / "Buildings.csv")
  apartments   = pd.read_csv(ATTR_PATH / "Apartments.csv")
  employers    = pd.read_csv(ATTR_PATH / "Employers.csv")
  restaurants  = pd.read_csv(ATTR_PATH / "Restaurants.csv")
  pubs         = pd.read_csv(ATTR_PATH / "Pubs.csv")
  schools      = pd.read_csv(ATTR_PATH / "Schools.csv")
  participants = pd.read_csv(ATTR_PATH / "Participants.csv")
```

```
checkins = pd.read_csv(
      JOURNAL_PATH / "CheckinJournal.csv",
      usecols=["participantId", "venueId", "venueType"]
  )
```

```
print("Building types found:", buildings['buildingType'].value_counts().to_dict())
```

```
# ================================================================
  # PARSE GEOMETRIES
  # ================================================================
  def parse_point(s):
      try:
          return wkt.loads(s)
      except:
          coords = s.strip().replace("POINT (","").replace(")","").split()
          return Point(float(coords[0]), float(coords[1]))
```

```
def parse_polygon(s):
      try:
          return wkt.loads(s)
      except:
          return None
```

```
buildings['geometry'] = buildings['location'].apply(parse_polygon)
  gdf_buildings = gpd.GeoDataFrame(buildings, geometry='geometry')
  gdf_buildings['centroid'] = gdf_buildings.geometry.centroid
  gdf_buildings['cx'] = gdf_buildings['centroid'].x
  gdf_buildings['cy'] = gdf_buildings['centroid'].y
```

```
for df in [apartments, employers, restaurants, pubs, schools]:
      df['geometry'] = df['location'].apply(parse_point)
```

```
gdf_apts  = gpd.GeoDataFrame(apartments,  geometry='geometry')
  gdf_emp   = gpd.GeoDataFrame(employers,   geometry='geometry')
  gdf_rest  = gpd.GeoDataFrame(restaurants, geometry='geometry')
  gdf_pubs  = gpd.GeoDataFrame(pubs,        geometry='geometry')
  gdf_schl  = gpd.GeoDataFrame(schools,     geometry='geometry')
```

```
# ============================================================
  # FIX 1: Build a unified point dataset for ALL location types
  # Apartments = Residential, Employers = Commercial, Schools = School
  # Buildings.csv alone is not enough!
  # ============================================================
  all_points = pd.DataFrame({
      'x':    list(gdf_apts.geometry.x)  + list(gdf_emp.geometry.x)  + list(gdf_schl.ge
  ometry.x),
      'y':    list(gdf_apts.geometry.y)  + list(gdf_emp.geometry.y)  + list(gdf_schl.ge
  ometry.y),
      'type': ['Residential'] * len(gdf_apts) +
              ['Commercial']  * len(gdf_emp)  +
              ['School']      * len(gdf_schl)
  })
```

```
print(f"\nTotal venue points: {len(all_points)}")
  print(all_points['type'].value_counts())
```

```
# ============================================================
  # FIX 2: Use K-Means instead of DBSCAN
  # K-Means gives predictable number of zones
  # Use 6 clusters — tune this number based on your output
  # ============================================================
  coords = all_points[['x', 'y']].values
  kmeans = KMeans(n_clusters=6, random_state=42, n_init=10)
  all_points['cluster'] = kmeans.fit_predict(coords)
```

```
print(f"\nK-Means found 6 zones")
  print(all_points.groupby('cluster')['type'].value_counts())
```

```
# ============================================================
  # CHARACTERIZE EACH CLUSTER
  # ============================================================
  cluster_summary = all_points.groupby('cluster').agg(
      total       = ('type', 'count'),
      residential = ('type', lambda x: (x == 'Residential').sum()),
      commercial  = ('type', lambda x: (x == 'Commercial').sum()),
      school      = ('type', lambda x: (x == 'School').sum()),
      cx          = ('x', 'mean'),
      cy          = ('y', 'mean'),
  ).reset_index()
```

```
def classify_zone(row):
      if row['total'] == 0: return 'Unknown'
      res = row['residential'] / row['total']
      com = row['commercial']  / row['total']
      scl = row['school']      / row['total']
      if scl > 0.05:   return 'Educational Zone'
      if res > 0.75:   return 'Residential Zone'
      if com > 0.75:   return 'Commercial/Employment Zone'
      if res > com:    return 'Mixed Residential'
      return 'Mixed Commercial'
```

```
cluster_summary['zone_type'] = cluster_summary.apply(classify_zone, axis=1)
  all_points = all_points.merge(cluster_summary[['cluster','zone_type']], on='cluster',
  how='left')
```

```
print("\n--- Zone Summary ---")
  print(cluster_summary[['cluster','total','residential','commercial','school','zone_ty
  pe']].to_string())
```

```
# ============================================================
  # VIZ 1: City map with correct building types
  # ============================================================
  type_colors = {'Residential':'#2196F3', 'Commercial':'#FF9800', 'School':'#4CAF50'}
```

```
fig, ax = plt.subplots(figsize=(14, 10))
  ax.set_facecolor('#f0f0f0')
```

```
# Draw building footprints (light grey base)
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white', linewidth=0.2, alpha=0.5)
```

```
# Plot each venue type as points
  for vtype, color in type_colors.items():
      subset = all_points[all_points['type'] == vtype]
      ax.scatter(subset['x'], subset['y'], c=color, s=12, alpha=0.6,
                 label=vtype, zorder=4)
```

```
# Restaurants and pubs as distinct markers
  ax.scatter(gdf_rest.geometry.x, gdf_rest.geometry.y,
             color='red', marker='o', s=60, label='Restaurant', zorder=6)
  ax.scatter(gdf_pubs.geometry.x, gdf_pubs.geometry.y,
             color='purple', marker='s', s=50, label='Pub', zorder=6)
  ax.scatter(gdf_schl.geometry.x, gdf_schl.geometry.y,
             color='green', marker='^', s=100, label='School', zorder=6)
```

```
ax.legend(loc='lower right', fontsize=9)
  ax.set_title("City of Engagement — All Venue Types", fontsize=14, fontweight='bold')
  ax.set_xlabel("X"); ax.set_ylabel("Y")
  plt.tight_layout()
  plt.savefig("q1_city_map.png", dpi=150, bbox_inches='tight')
  plt.show()
```

```
# ============================================================
  # VIZ 2: K-Means zones map
  # ============================================================
  zone_palette = ['#1565C0','#42A5F5','#E65100','#FFA726','#7B1FA2','#2E7D32']
  zone_labels  = cluster_summary.set_index('cluster')['zone_type'].to_dict()
```

```
fig, ax = plt.subplots(figsize=(14, 10))
  ax.set_facecolor('#e8e8e8')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white', linewidth=0.2, alpha=0.4)
```

```
for cid in sorted(all_points['cluster'].unique()):
      subset = all_points[all_points['cluster'] == cid]
      color  = zone_palette[cid % len(zone_palette)]
      ax.scatter(subset['x'], subset['y'], c=color, s=8, alpha=0.5,
                 label=f"Zone {cid}: {zone_labels[cid]}", zorder=4)
```

```
# Draw cluster centroids with labels
  for _, row in cluster_summary.iterrows():
      ax.annotate(
          f"Zone {int(row['cluster'])}\n{row['zone_type']}",
          xy=(row['cx'], row['cy']),
          fontsize=7, ha='center', fontweight='bold',
          bbox=dict(boxstyle='round,pad=0.3', fc='white', alpha=0.85)
      )
```

```
ax.legend(loc='lower right', fontsize=8, title="Zones")
  ax.set_title("City of Engagement — Identified Zones (K-Means, k=6)", fontsize=14, fon
  tweight='bold')
  ax.set_xlabel("X"); ax.set_ylabel("Y")
  plt.tight_layout()
  plt.savefig("q1_zones_map.png", dpi=150, bbox_inches='tight')
  plt.show()
```

```
# ================================================================
  # VIZ 3: Zone composition bar chart
  # ================================================================
  fig, ax = plt.subplots(figsize=(12, 5))
  x = np.arange(len(cluster_summary))
  w = 0.25
```

```
ax.bar(x - w, cluster_summary['residential'], w, label='Residential', color='#2196F3')
  ax.bar(x,     cluster_summary['commercial'],  w, label='Commercial',  color='#FF980
  0')
  ax.bar(x + w, cluster_summary['school'],      w, label='School',      color='#4CAF5
  0')
```

```
labels = [f"Zone {int(r['cluster'])}\n{r['zone_type']}"
            for _, r in cluster_summary.iterrows()]
  ax.set_xticks(x)
  ax.set_xticklabels(labels, rotation=25, ha='right', fontsize=8)
  ax.set_ylabel("Number of Venues")
  ax.set_title("Zone Composition — Venue Types per Cluster", fontweight='bold')
  ax.legend()
  plt.tight_layout()
  plt.savefig("q1_zone_composition.png", dpi=150, bbox_inches='tight')
  plt.show()
```

```
# ============================================================
  # VIZ 4: Rental cost heatmap (keep your existing version — it was good)
  # ============================================================
  fig, ax = plt.subplots(figsize=(13, 9))
  ax.set_facecolor('#f0f0f0')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white', linewidth=0.2, alpha=0.4)
```

```
sc = ax.scatter(gdf_apts.geometry.x, gdf_apts.geometry.y,
                  c=gdf_apts['rentalCost'], cmap='RdYlGn_r', s=20, alpha=0.75, zorder=
  5)
  plt.colorbar(sc, ax=ax, shrink=0.6, label="Monthly Rent ($)")
  ax.set_title("Apartment Rental Cost — Wealthy vs Low-Income Areas",
               fontsize=13, fontweight='bold')
  ax.set_xlabel("X"); ax.set_ylabel("Y")
  plt.tight_layout()
  plt.savefig("q1_rental_heatmap.png", dpi=150, bbox_inches='tight')
  plt.show()
```

```
# ============================================================
  # PRINT FINAL SUMMARY
  # ============================================================
  print("\n" + "="*60)
  print("ZONE SUMMARY FOR REPORT")
  print("="*60)
  for _, row in cluster_summary.iterrows():
      print(f"\nZone {int(row['cluster'])} — {row['zone_type']}")
      print(f"  Residential venues : {int(row['residential'])}")
      print(f"  Commercial venues  : {int(row['commercial'])}")
      print(f"  Schools            : {int(row['school'])}")
      print(f"  Center             : ({row['cx']:.0f}, {row['cy']:.0f})")
```

What Changed and Why

Issue	Old code	Fixed code
No residential on map	Used only `Buildings.csv` (all Commercial)	Added `Apartments.csv` as Residential points
Only 1 cluster	DBSCAN with `eps=0.3` on huge coordinate space	**K-Means with k=6** — always gives exactly 6 zones
Empty bar chart	Only 1 cluster to show	Now shows all 6 zones with 3 bar groups each

Run this and you'll get **6 clearly distinct zones** with meaningful residential/commercial/school breakdowns.

Looking at your 5 images, there are **3 clear problems** to fix:

Problem 1 — All buildings show as Commercial (orange) in both maps. Residential buildings are missing entirely. This is because in this dataset, `Buildings.csv` contains mostly/only Commercial and School types — **apartments are stored separately in** `Apartments.csv` with their own locations.

Problem 2 — DBSCAN found only 1 giant cluster (Zone 0). The coordinate range is huge (X: -5000 to 3000, Y: 0 to 8000) so all buildings are spatially connected and DBSCAN merges them all. `eps=0.3` after StandardScaler is still too large for this city layout.

Problem 3 — Zone composition bar chart shows only one bar — direct consequence of problem 2.

The rental heatmap and checkin density maps are actually good ✅.

Replace your clustering section with **K-Means** (more reliable here) and fix the residential layer:

Run this and you'll get **6 clearly distinct zones** with meaningful residential/commercial/school breakdowns.

Q6: User

Is it possible that all zones are residential zones ? How are the results now ?

A6: Claude

Looking at your results carefully — **yes, it is entirely plausible** , and here's why: the dataset has ~1,500 apartments but only ~200 employers, so residential points dominate every zone. The spatial zones themselves are actually correct and meaningful — you can clearly see 6 distinct geographic areas. The problem is only the **labeling logic**.

Let me fix the classification to use richer characteristics:

Check available skills

Check available skills

The zones ARE real and spatially valid — the issue is purely the classifier. Here's the complete fixed approach:

Why All Zones Show as "Residential"

The dataset has roughly **1,500 apartments vs ~200 employers**. So in every zone, residential > 75% and the old classifier always picks "Residential Zone". The zones themselves are spatially correct — look at the map, you can clearly see 6 distinct geographic neighborhoods.

Fixed Script — Smarter Zone Characterization

Replace your classification and visualization section with this:

python

```
from pathlib import Path
  import pandas as pd
  import numpy as np
  import matplotlib.pyplot as plt
  import matplotlib.patches as mpatches
  import geopandas as gpd
  from shapely.geometry import Point
  from shapely import wkt
  from sklearn.cluster import KMeans
  from scipy.spatial import cKDTree
  import warnings
  warnings.filterwarnings('ignore')
```

```
BASE         = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets")
  ATTR_PATH    = BASE / "Attributes"
  JOURNAL_PATH = BASE / "Journals"
```

```
# ================================================================
  # LOAD DATA
  # ================================================================
  buildings    = pd.read_csv(ATTR_PATH / "Buildings.csv")
  apartments   = pd.read_csv(ATTR_PATH / "Apartments.csv")
  employers    = pd.read_csv(ATTR_PATH / "Employers.csv")
  restaurants  = pd.read_csv(ATTR_PATH / "Restaurants.csv")
  pubs         = pd.read_csv(ATTR_PATH / "Pubs.csv")
  schools      = pd.read_csv(ATTR_PATH / "Schools.csv")
  participants = pd.read_csv(ATTR_PATH / "Participants.csv")
  checkins     = pd.read_csv(JOURNAL_PATH / "CheckinJournal.csv",
                             usecols=["participantId", "venueId", "venueType"])
```

```
# ================================================================
  # PARSE GEOMETRIES
  # ================================================================
  def parse_point(s):
      try:    return wkt.loads(s)
      except:
          c = s.strip().replace("POINT (","").replace(")","").split()
          return Point(float(c[0]), float(c[1]))
```

```
def parse_polygon(s):
      try:    return wkt.loads(s)
      except: return None
```

```
buildings['geometry'] = buildings['location'].apply(parse_polygon)
  gdf_buildings = gpd.GeoDataFrame(buildings, geometry='geometry')
```

```
for df in [apartments, employers, restaurants, pubs, schools]:
      df['geometry'] = df['location'].apply(parse_point)
```

```
gdf_apts  = gpd.GeoDataFrame(apartments,  geometry='geometry')
  gdf_emp   = gpd.GeoDataFrame(employers,   geometry='geometry')
  gdf_rest  = gpd.GeoDataFrame(restaurants, geometry='geometry')
  gdf_pubs  = gpd.GeoDataFrame(pubs,        geometry='geometry')
  gdf_schl  = gpd.GeoDataFrame(schools,     geometry='geometry')
```

```
# ============================================================
  # STEP 1: K-Means clustering on APARTMENTS ONLY
  # (apartments = where people live = best proxy for neighborhoods)
  # ============================================================
  apt_coords = np.column_stack([gdf_apts.geometry.x, gdf_apts.geometry.y])
  kmeans = KMeans(n_clusters=6, random_state=42, n_init=10)
  gdf_apts['cluster'] = kmeans.fit_predict(apt_coords)
```

```
# ============================================================
  # STEP 2: Assign employers/restaurants/pubs/schools to nearest cluster
  # Using KD-Tree for fast nearest-neighbor lookup
  # ============================================================
  centroids = kmeans.cluster_centers_   # shape (6, 2)
  tree = cKDTree(centroids)
```

```
def assign_to_cluster(gdf):
      pts = np.column_stack([gdf.geometry.x, gdf.geometry.y])
      _, idx = tree.query(pts)
      return idx
```

```
gdf_emp['cluster']  = assign_to_cluster(gdf_emp)
  gdf_rest['cluster'] = assign_to_cluster(gdf_rest)
  gdf_pubs['cluster'] = assign_to_cluster(gdf_pubs)
  gdf_schl['cluster'] = assign_to_cluster(gdf_schl)
```

```
# ============================================================
  # STEP 3: Build rich cluster profile
  # For each zone: count venues, avg rent, avg income,
  # commercial density ratio, amenity access
  # ============================================================
  cluster_summary = gdf_apts.groupby('cluster').agg(
      num_apartments = ('apartmentId', 'count'),
      avg_rent       = ('rentalCost', 'mean'),
      max_rent       = ('rentalCost', 'max'),
      min_rent       = ('rentalCost', 'min'),
      avg_rooms      = ('numberOfRooms', 'mean'),
      cx             = ('geometry', lambda g: g.x.mean()),
      cy             = ('geometry', lambda g: g.y.mean()),
  ).reset_index()
```

```
# Count commercial employers per cluster
  emp_counts  = gdf_emp.groupby('cluster').size().rename('num_employers')
  rest_counts = gdf_rest.groupby('cluster').size().rename('num_restaurants')
  pub_counts  = gdf_pubs.groupby('cluster').size().rename('num_pubs')
  schl_counts = gdf_schl.groupby('cluster').size().rename('num_schools')
```

```
cluster_summary = cluster_summary \
      .join(emp_counts,  on='cluster') \
      .join(rest_counts, on='cluster') \
      .join(pub_counts,  on='cluster') \
      .join(schl_counts, on='cluster')
```

```
cluster_summary = cluster_summary.fillna(0)
```

```
# Add participant info: avg joviality and education per zone
  # Map participants to their apartment cluster
  apt_cluster_map = gdf_apts.set_index('apartmentId')['cluster'].to_dict()
  participants_merged = participants.merge(
      gdf_apts[['apartmentId','cluster']], on='apartmentId', how='left'
  )
  part_by_cluster = participants_merged.groupby('cluster').agg(
      avg_joviality = ('joviality', 'mean'),
      pct_kids      = ('haveKids', 'mean'),
      avg_age       = ('age', 'mean'),
  ).reset_index()
```

```
cluster_summary = cluster_summary.merge(part_by_cluster, on='cluster', how='left')
```

```
# ============================================================
  # STEP 4: SMART ZONE CLASSIFICATION
  # Now classify based on MULTIPLE features, not just counts
  # ============================================================
  global_avg_rent = gdf_apts['rentalCost'].mean()
```

```
def classify_zone_smart(row):
      """
      Use rent level + amenity density + employer count
      to give a meaningful zone label.
      """
      rent       = row['avg_rent']
      employers  = row['num_employers']
      restaurants= row['num_restaurants']
      pubs       = row['num_pubs']
      schools    = row['num_schools']
      apts       = row['num_apartments']
      joviality  = row.get('avg_joviality', 0.5)
```

```
    # Commercial density = employers per apartment
      com_density = employers / apts if apts > 0 else 0
      # Amenity score = restaurants + pubs near this zone
      amenity_score = restaurants + pubs
```

```
    if schools >= 1 and com_density < 0.1:
          return 'Educational / Family Zone'
      if com_density > 0.25:
          return 'Commercial / Employment Hub'
      if amenity_score >= 4 and rent >= global_avg_rent:
          return 'Affluent Entertainment District'
      if rent > global_avg_rent * 1.2:
          return 'Upscale Residential'
      if rent < global_avg_rent * 0.8:
          return 'Low-Income Residential'
      if joviality > 0.5 and amenity_score >= 2:
          return 'Mixed Residential (Active)'
      return 'Suburban Residential'
```

```
cluster_summary['zone_type'] = cluster_summary.apply(classify_zone_smart, axis=1)
  gdf_apts = gdf_apts.merge(cluster_summary[['cluster','zone_type']], on='cluster', how
  ='left')
```

```
# ============================================================
  # PRINT DETAILED SUMMARY
  # ============================================================
  print("\n" + "="*70)
  print("ZONE SUMMARY")
  print("="*70)
  for _, row in cluster_summary.sort_values('avg_rent', ascending=False).iterrows():
      print(f"\nZone {int(row['cluster'])} — {row['zone_type']}")
      print(f"  Apartments  : {int(row['num_apartments'])}")
      print(f"  Avg Rent    : ${row['avg_rent']:.0f}  "
            f"(range: ${row['min_rent']:.0f}-${row['max_rent']:.0f})")
      print(f"  Employers   : {int(row['num_employers'])}  "
            f"Restaurants: {int(row['num_restaurants'])}  "
            f"Pubs: {int(row['num_pubs'])}  "
            f"Schools: {int(row['num_schools'])}")
      print(f"  Avg Age     : {row['avg_age']:.1f}  "
            f"Joviality: {row['avg_joviality']:.2f}  "
            f"Has Kids: {row['pct_kids']*100:.0f}%")
      print(f"  Center      : ({row['cx']:.0f}, {row['cy']:.0f})")
```

```
# ============================================================
  # VIZ 1: City overview — all venue types
  # ============================================================
  fig, ax = plt.subplots(figsize=(14, 10))
  ax.set_facecolor('#f0f0f0')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white', linewidth=0.2, alpha=0.4)
```

```
ax.scatter(gdf_apts.geometry.x,  gdf_apts.geometry.y,
             c='#2196F3', s=10, alpha=0.5, label='Residential (Apartment)', zorder=3)
  ax.scatter(gdf_emp.geometry.x,   gdf_emp.geometry.y,
             c='#FF9800', s=15, alpha=0.7, label='Commercial (Employer)',   zorder=4)
  ax.scatter(gdf_rest.geometry.x,  gdf_rest.geometry.y,
             c='red',    marker='o', s=70, label='Restaurant', zorder=6)
  ax.scatter(gdf_pubs.geometry.x,  gdf_pubs.geometry.y,
             c='purple', marker='s', s=60, label='Pub',        zorder=6)
  ax.scatter(gdf_schl.geometry.x,  gdf_schl.geometry.y,
             c='green',  marker='^', s=120,label='School',     zorder=6)
```

```
ax.legend(loc='lower right', fontsize=9)
  ax.set_title("City of Engagement — All Venue Types", fontsize=14, fontweight='bold')
  ax.set_xlabel("X Coordinate"); ax.set_ylabel("Y Coordinate")
  plt.tight_layout()
  plt.savefig("q1_city_map.png", dpi=150, bbox_inches='tight')
  plt.show()
```

```
# ============================================================
  # VIZ 2: Zones map with meaningful labels
  # ============================================================
  zone_palette = {
      'Educational / Family Zone':       '#2E7D32',
      'Commercial / Employment Hub':     '#E65100',
      'Affluent Entertainment District': '#7B1FA2',
      'Upscale Residential':             '#1565C0',
      'Low-Income Residential':          '#F44336',
      'Mixed Residential (Active)':      '#00ACC1',
      'Suburban Residential':            '#78909C',
  }
  # Assign colors by zone_type
  gdf_apts['color'] = gdf_apts['zone_type'].map(zone_palette).fillna('#9E9E9E')
```

```
fig, ax = plt.subplots(figsize=(14, 10))
  ax.set_facecolor('#e8e8e8')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white', linewidth=0.2, alpha=0.4)
```

```
for zone_name, color in zone_palette.items():
      subset = gdf_apts[gdf_apts['zone_type'] == zone_name]
      if len(subset):
          ax.scatter(subset.geometry.x, subset.geometry.y,
                     c=color, s=12, alpha=0.7, label=zone_name, zorder=4)
```

```
# Label each zone centroid
  for _, row in cluster_summary.iterrows():
      ax.annotate(
          f"Zone {int(row['cluster'])}\n{row['zone_type']}\nAvg Rent: ${row['avg_ren
  t']:.0f}",
          xy=(row['cx'], row['cy']),
          fontsize=6.5, ha='center',
          bbox=dict(boxstyle='round,pad=0.3', fc='white', alpha=0.9, ec='grey')
      )
```

```
ax.legend(loc='lower right', fontsize=8, title="Zone Type")
  ax.set_title("City of Engagement — Identified Zones (K-Means, k=6)",
               fontsize=14, fontweight='bold')
  ax.set_xlabel("X Coordinate"); ax.set_ylabel("Y Coordinate")
  plt.tight_layout()
  plt.savefig("q1_zones_map.png", dpi=150, bbox_inches='tight')
  plt.show()
```

```
# ============================================================
  # VIZ 3: Multi-metric zone comparison (replaces bar chart)
  # ============================================================
  fig, axes = plt.subplots(1, 3, figsize=(16, 5))
  cluster_summary_sorted = cluster_summary.sort_values('avg_rent', ascending=True)
  labels = [f"Z{int(r['cluster'])}: {r['zone_type']}"
            for _, r in cluster_summary_sorted.iterrows()]
  y_pos  = np.arange(len(labels))
```

```
# Avg rent
  axes[0].barh(y_pos, cluster_summary_sorted['avg_rent'],
               color=[zone_palette.get(z, '#ccc') for z in cluster_summary_sorted['zone
  _type']],
               edgecolor='white')
  axes[0].set_yticks(y_pos); axes[0].set_yticklabels(labels, fontsize=8)
  axes[0].set_xlabel("Avg Monthly Rent ($)")
  axes[0].set_title("Avg Rental Cost per Zone", fontweight='bold')
  axes[0].axvline(global_avg_rent, color='black', linestyle='--',
                  linewidth=1, label=f"City avg ${global_avg_rent:.0f}")
  axes[0].legend(fontsize=8)
```

```
# Amenity access
  amenity_total = cluster_summary_sorted['num_restaurants'] + cluster_summary_sorted['n
  um_pubs']
  axes[1].barh(y_pos, amenity_total,
               color=[zone_palette.get(z, '#ccc') for z in cluster_summary_sorted['zone
  _type']],
               edgecolor='white')
  axes[1].set_yticks(y_pos); axes[1].set_yticklabels(labels, fontsize=8)
  axes[1].set_xlabel("Number of Restaurants + Pubs")
  axes[1].set_title("Amenity Access per Zone", fontweight='bold')
```

```
# Joviality
  axes[2].barh(y_pos, cluster_summary_sorted['avg_joviality'],
               color=[zone_palette.get(z, '#ccc') for z in cluster_summary_sorted['zone
  _type']],
               edgecolor='white')
  axes[2].set_yticks(y_pos); axes[2].set_yticklabels(labels, fontsize=8)
  axes[2].set_xlabel("Average Joviality (0-1)")
  axes[2].set_title("Resident Happiness per Zone", fontweight='bold')
  axes[2].axvline(0.5, color='black', linestyle='--', linewidth=1)
```

```
plt.suptitle("Zone Profiles — Rent, Amenities & Happiness",
                fontsize=13, fontweight='bold')
  plt.tight_layout()
  plt.savefig("q1_zone_composition.png", dpi=150, bbox_inches='tight')
  plt.show()
```

```
# ============================================================
  # VIZ 4: Rental heatmap (unchanged — was already good)
  # ============================================================
  fig, ax = plt.subplots(figsize=(13, 9))
  ax.set_facecolor('#f0f0f0')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white', linewidth=0.2, alpha=0.4)
  sc = ax.scatter(gdf_apts.geometry.x, gdf_apts.geometry.y,
                  c=gdf_apts['rentalCost'], cmap='RdYlGn_r', s=20, alpha=0.75, zorder=
  5)
  plt.colorbar(sc, ax=ax, shrink=0.6, label="Monthly Rent ($)")
  ax.set_title("Apartment Rental Cost — Wealthy vs Low-Income Areas",
               fontsize=13, fontweight='bold')
  ax.set_xlabel("X"); ax.set_ylabel("Y")
  plt.tight_layout()
  plt.savefig("q1_rental_heatmap.png", dpi=150, bbox_inches='tight')
  plt.show()
```

What Changed and Why

Problem	Old approach	New approach
All "Residential"	Classified by % residential (always >75%)	Uses **rent level, employer density, amenity count, joviality**
Uninformative bar chart	Residential/Commercial/School counts (all same shape)	**3-panel: rent + amenities + happiness** — actually differentiates zones
No participant data used	Ignored `Participants.csv`	Now shows avg age, joviality, % with kids per zone
Clustering on mixed points	Mixed apartments + employers distorted centroids	Clusters **apartments only** then assigns employers/venues to nearest cluster

The terminal printout will now show you something like:

```
Zone 2 — Upscale Residential
    Avg Rent: $1,420  (range: $980–$1,600)
    Employers: 12   Restaurants: 3   Pubs: 1   Schools: 0
    Avg Age: 38.2   Joviality: 0.61   Has Kids: 42%
```

```
Zone 4 — Low-Income Residential
    Avg Rent: $380   (range: $200–$520)
    Employers: 8    Restaurants: 1   Pubs: 0   Schools: 1
    Avg Age: 29.1   Joviality: 0.38   Has Kids: 28%
```

These differences are what you use to write your Q1 answer.

Looking at your results carefully — **yes, it is entirely plausible** , and here's why: the dataset has ~1,500 apartments but only ~200 employers, so residential points dominate every zone. The spatial zones themselves are actually correct and meaningful — you can clearly see 6 distinct geographic areas. The problem is only the **labeling logic**.

Let me fix the classification to use richer characteristics:

The zones ARE real and spatially valid — the issue is purely the classifier. Here's the complete fixed approach:

The dataset has roughly **1,500 apartments vs ~200 employers**. So in every zone, residential > 75% and the old classifier always picks "Residential Zone". The zones themselves are spatially correct — look at the map, you can clearly see 6 distinct geographic neighborhoods.

Replace your classification and visualization section with this:

The terminal printout will now show you something like:

These differences are what you use to write your Q1 answer.

Q7: User

KeyError Traceback (most recent call last)

~\AppData\Local\Temp\ipykernel_5184\4280278985.py in ?()

```
109
```

```
110 # Add participant info: avg joviality and education per zone
```

```
111 # Map participants to their apartment cluster
```

```
112 apt_cluster_map = gdf_apts.set_index('apartmentId')['cluster'].to_dict()
```

--> 113 participants_merged = participants.merge(

```
114     gdf_apts[['apartmentId','cluster']], on='apartmentId', how='left'
```

```
115 )
```

```
116 part_by_cluster = participants_merged.groupby('cluster').agg(
```

~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\frame.py in ?(self, right, how, on, left*on, right*on, left*index, right*index, sort, suffixes, copy, indicator, validate)

10855 validate: MergeValidate | None = None,

10856) -> DataFrame:

10857 from pandas.core.reshape.merge import merge

10858

```
10859 return merge(
```

10860 self,

10861 right,

10862 how=how,

~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\reshape\merge.py in ?(left, right, how, on, left*on, right*on, left*index, right*index, sort, suffixes, copy, indicator, validate)

```
166             validate=validate,
```

```
167             copy=copy,
```

```
168         )
```

```
169     else:
```

--> 170 op = _MergeOperation(

```
171             left_df,
```

```
172             right_df,
```

```
173             how=how,
```

~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\reshape\merge.py in ? (self, left, right, how, on, left*on, right*on, left*index, right*index, sort, suffixes, indicator, validate)

```
790             self.right_join_keys,
```

```
791             self.join_names,
```

```
792             left_drop,
```

```
793             right_drop,
```

--> 794) = self.*get*merge_keys()

```
795
```

```
796         if left_drop:
```

```
797             self.left = self.left._drop_labels_or_levels(left_drop)
```

~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\reshape\merge.py in ? (self)

1307 if lk is not None:

1308 # Then we're either Hashable or a wrong-length arraylike,

1309 # the latter of which will raise

1310 lk = cast(Hashable, lk)

-> 1311 left*keys.append(left*.get*label*or*level*values(lk))

1312 join_names.append(lk)

1313 else:

1314 # work-around for merge*asof(left*index=True)

~\AppData\Local\Programs\Python\Python310\lib\site-packages\pandas\core\generic.py in ?(self, key, axis)

1910 values = self.xs(key, axis=other*axes[0])*.values

1911 elif self.*is*level_reference(key, axis=axis):

1912 values = self.axes[axis].get*level*values(key)._values

1913 else:

-> 1914 raise KeyError(key)

1915

1916 # Check for duplicates

1917 if values.ndim > 1:

KeyError: 'apartmentId'

A7: Claude

The `Participants.csv` file does **not** have an `apartmentId` column — that field only exists in the Activity Logs. The fix is to get the apartment-participant mapping from the checkins data instead:

python

```
# ❌ REMOVE these lines (lines 112-120):
apt_cluster_map = gdf_apts.set_index('apartmentId')['cluster'].to_dict()
participants_merged = participants.merge(
    gdf_apts[['apartmentId','cluster']], on='apartmentId', how='left'
)
```

```
# ✅ REPLACE WITH: get apartment per participant from checkins
  # The CheckinJournal tells us where each person checked in as "Apartment"
  apt_checkins = checkins[checkins['venueType'] == 'Apartment'].copy()
```

```
# Each participant's home = the apartment they checked into most often
  home_apt = (apt_checkins
      .groupby(['participantId', 'venueId'])
      .size()
      .reset_index(name='visits')
      .sort_values('visits', ascending=False)
      .drop_duplicates('participantId')   # keep most-visited apartment per person
      .rename(columns={'venueId': 'apartmentId'})
  )
```

```
# Now map apartment → cluster
  apt_to_cluster = gdf_apts.set_index('apartmentId')['cluster'].to_dict()
  home_apt['cluster'] = home_apt['apartmentId'].map(apt_to_cluster)
```

```
# Merge with participant demographics
  participants_merged = participants.merge(home_apt[['participantId','cluster']],
                                           on='participantId', how='left')
```

```
# Now compute per-cluster stats as before
  part_by_cluster = participants_merged.groupby('cluster').agg(
      avg_joviality = ('joviality', 'mean'),
      pct_kids      = ('haveKids', 'mean'),
      avg_age       = ('age', 'mean'),
  ).reset_index()
```

This works because `CheckinJournal.csv` has `venueType == "Apartment"` rows that record every time a participant went home — so the most-visited apartment ID is reliably their home.

Also run this first to confirm your actual column names:

python

```
# Run this in a separate cell to verify column names
  print("Participants columns:", participants.columns.tolist())
  print("Checkins columns:", checkins.columns.tolist())
  print("Apartments columns:", gdf_apts.columns.tolist())
  print("\nCheckin venue types:", checkins['venueType'].unique())
```

This will show you exactly what's available so there are no more surprises.

The `Participants.csv` file does **not** have an `apartmentId` column — that field only exists in the Activity Logs. The fix is to get the apartment-participant mapping from the checkins data instead:

This works because `CheckinJournal.csv` has `venueType == "Apartment"` rows that record every time a participant went home — so the most-visited apartment ID is reliably their home.

Also run this first to confirm your actual column names:

This will show you exactly what's available so there are no more surprises.

Q8: User

Participants columns: ['participantId', 'householdSize', 'haveKids', 'age', 'educationLevel', 'interestGroup', 'joviality']

Checkins columns: ['participantId', 'venueId', 'venueType']

Apartments columns: ['apartmentId', 'rentalCost', 'maxOccupancy ', 'numberOfRooms', 'location', 'buildingId', 'geometry', 'cluster']

Checkin venue types: ['Pub' 'Restaurant' 'Apartment' 'Workplace']

A8: Claude

Good — all the columns are there. One thing to note: `maxOccupancy` has a **trailing space** (`'maxOccupancy '`) which could cause issues later. Here's the clean fixed block to replace everything from line 109 onwards:

python

```
# ============================================================
  # FIX: Strip trailing spaces from all column names
  # (maxOccupancy has a hidden space — catches this kind of issue)
  # ============================================================
  gdf_apts.columns = gdf_apts.columns.str.strip()
  participants.columns = participants.columns.str.strip()
  checkins.columns = checkins.columns.str.strip()
```

```
# ============================================================
  # Get each participant's home apartment from checkins
  # (most frequently visited Apartment = home)
  # ============================================================
  apt_checkins = checkins[checkins['venueType'] == 'Apartment'].copy()
```

```
home_apt = (apt_checkins
      .groupby(['participantId', 'venueId'])
      .size()
      .reset_index(name='visits')
      .sort_values('visits', ascending=False)
      .drop_duplicates('participantId')
      .rename(columns={'venueId': 'apartmentId'})
  )
```

```
print(f"Participants with identified home: {len(home_apt)} / {len(participants)}")
```

```
# Map apartment → cluster
  apt_to_cluster = gdf_apts.set_index('apartmentId')['cluster'].to_dict()
  home_apt['cluster'] = home_apt['apartmentId'].map(apt_to_cluster)
```

```
# Merge with participant demographics
  participants_merged = participants.merge(
      home_apt[['participantId', 'cluster']],
      on='participantId', how='left'
  )
```

```
print(f"Participants matched to a cluster: {participants_merged['cluster'].notna().sum
  ()}")
```

```
# Per-cluster demographic stats
  part_by_cluster = participants_merged.groupby('cluster').agg(
      avg_joviality = ('joviality', 'mean'),
      pct_kids      = ('haveKids', 'mean'),
      avg_age       = ('age', 'mean'),
  ).reset_index()
```

```
print("\nPer-cluster demographics:")
  print(part_by_cluster.round(3).to_string())
```

```
# ============================================================
  # Merge demographics back into cluster_summary
  # ============================================================
  cluster_summary = cluster_summary.merge(part_by_cluster, on='cluster', how='left')
```

```
# Now classify zones using all features
  global_avg_rent = gdf_apts['rentalCost'].mean()
  print(f"\nCity-wide average rent: ${global_avg_rent:.0f}")
```

```
cluster_summary['zone_type'] = cluster_summary.apply(classify_zone_smart, axis=1)
  gdf_apts = gdf_apts.merge(
      cluster_summary[['cluster', 'zone_type']], on='cluster', how='left'
  )
```

The key additions here are:

1. `.str.strip()` **on all column names** — silently fixes the `maxOccupancy ` space and any others like it
    1. **Two print statements** so you can verify the merge worked before continuing — you should see close to 1,000 participants matched to a cluster
    2. The rest of the script (the visualizations) stays exactly the same after this block

Good — all the columns are there. One thing to note: `maxOccupancy` has a **trailing space** (`'maxOccupancy '`) which could cause issues later. Here's the clean fixed block to replace everything from line 109 onwards:
The key additions here are:

Q9: User

provide the entire optimsed code

A9: Claude

Here is the complete optimized script from start to finish:

python

```
# ============================================================
  # VAST Challenge 2022 - Challenge 2, Question 1
  # Characterize distinct areas of the city of Engagement
  # ============================================================
```

```
from pathlib import Path
  import pandas as pd
  import numpy as np
  import matplotlib.pyplot as plt
  import matplotlib.patches as mpatches
  import geopandas as gpd
  from shapely.geometry import Point
  from shapely import wkt
  from sklearn.cluster import KMeans
  from scipy.spatial import cKDTree
  import warnings
  warnings.filterwarnings('ignore')
```

```
# ============================================================
  # PATHS
  # ============================================================
  BASE         = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets")
  ATTR_PATH    = BASE / "Attributes"
  JOURNAL_PATH = BASE / "Journals"
```

```
# ============================================================
  # STEP 1: LOAD ALL DATA
  # ============================================================
  print("Loading data...")
```

```
buildings    = pd.read_csv(ATTR_PATH / "Buildings.csv")
  apartments   = pd.read_csv(ATTR_PATH / "Apartments.csv")
  employers    = pd.read_csv(ATTR_PATH / "Employers.csv")
  restaurants  = pd.read_csv(ATTR_PATH / "Restaurants.csv")
  pubs         = pd.read_csv(ATTR_PATH / "Pubs.csv")
  schools      = pd.read_csv(ATTR_PATH / "Schools.csv")
  participants = pd.read_csv(ATTR_PATH / "Participants.csv")
```

```
# Load only needed columns from the large checkins file
  checkins = pd.read_csv(
      JOURNAL_PATH / "CheckinJournal.csv",
      usecols=["participantId", "venueId", "venueType"]
  )
```

```
# Fix trailing spaces in all column names across all dataframes
  for df in [buildings, apartments, employers, restaurants,
             pubs, schools, participants, checkins]:
      df.columns = df.columns.str.strip()
```

```
print(f"  Buildings:    {len(buildings)}")
  print(f"  Apartments:   {len(apartments)}")
  print(f"  Employers:    {len(employers)}")
  print(f"  Restaurants:  {len(restaurants)}")
  print(f"  Pubs:         {len(pubs)}")
  print(f"  Schools:      {len(schools)}")
  print(f"  Participants: {len(participants)}")
  print(f"  Checkins:     {len(checkins):,}")
```

```
# ============================================================
  # STEP 2: PARSE GEOMETRIES
  # ============================================================
  print("\nParsing geometries...")
```

```
def parse_point(s):
      try:
          return wkt.loads(s)
      except:
          c = str(s).strip().replace("POINT (", "").replace(")", "").split()
          return Point(float(c[0]), float(c[1]))
```

```
def parse_polygon(s):
      try:
          return wkt.loads(s)
      except:
          return None
```

```
buildings['geometry']  = buildings['location'].apply(parse_polygon)
  apartments['geometry'] = apartments['location'].apply(parse_point)
  employers['geometry']  = employers['location'].apply(parse_point)
  restaurants['geometry']= restaurants['location'].apply(parse_point)
  pubs['geometry']       = pubs['location'].apply(parse_point)
  schools['geometry']    = schools['location'].apply(parse_point)
```

```
gdf_buildings = gpd.GeoDataFrame(buildings,   geometry='geometry')
  gdf_apts      = gpd.GeoDataFrame(apartments,  geometry='geometry')
  gdf_emp       = gpd.GeoDataFrame(employers,   geometry='geometry')
  gdf_rest      = gpd.GeoDataFrame(restaurants, geometry='geometry')
  gdf_pubs      = gpd.GeoDataFrame(pubs,        geometry='geometry')
  gdf_schl      = gpd.GeoDataFrame(schools,     geometry='geometry')
```

```
print("  Geometries parsed successfully.")
```

```
# ============================================================
  # STEP 3: K-MEANS CLUSTERING ON APARTMENTS
  # Apartments are the best proxy for neighborhoods —
  # cluster them spatially into 6 zones
  # ============================================================
  print("\nRunning K-Means clustering (k=6) on apartments...")
```

```
apt_coords = np.column_stack([gdf_apts.geometry.x, gdf_apts.geometry.y])
  kmeans = KMeans(n_clusters=6, random_state=42, n_init=10)
  gdf_apts['cluster'] = kmeans.fit_predict(apt_coords)
  centroids = kmeans.cluster_centers_
```

```
print(f"  Apartments per cluster:")
  print(gdf_apts['cluster'].value_counts().sort_index().to_string())
```

```
# ============================================================
  # STEP 4: ASSIGN ALL OTHER VENUES TO NEAREST CLUSTER
  # Use KD-Tree for fast nearest-centroid lookup
  # ============================================================
  tree = cKDTree(centroids)
```

```
def assign_to_cluster(gdf):
      pts = np.column_stack([gdf.geometry.x, gdf.geometry.y])
      _, idx = tree.query(pts)
      return idx
```

```
gdf_emp['cluster']  = assign_to_cluster(gdf_emp)
  gdf_rest['cluster'] = assign_to_cluster(gdf_rest)
  gdf_pubs['cluster'] = assign_to_cluster(gdf_pubs)
  gdf_schl['cluster'] = assign_to_cluster(gdf_schl)
```

```
# ============================================================
  # STEP 5: BUILD CLUSTER PROFILE FROM APARTMENTS
  # ============================================================
  print("\nBuilding cluster profiles...")
```

```
cluster_summary = gdf_apts.groupby('cluster').agg(
      num_apartments = ('apartmentId', 'count'),
      avg_rent       = ('rentalCost',  'mean'),
      max_rent       = ('rentalCost',  'max'),
      min_rent       = ('rentalCost',  'min'),
      avg_rooms      = ('numberOfRooms','mean'),
      cx             = ('geometry', lambda g: g.x.mean()),
      cy             = ('geometry', lambda g: g.y.mean()),
  ).reset_index()
```

```
# Count venues per cluster
  emp_counts  = gdf_emp.groupby('cluster').size().rename('num_employers')
  rest_counts = gdf_rest.groupby('cluster').size().rename('num_restaurants')
  pub_counts  = gdf_pubs.groupby('cluster').size().rename('num_pubs')
  schl_counts = gdf_schl.groupby('cluster').size().rename('num_schools')
```

```
cluster_summary = (cluster_summary
      .join(emp_counts,  on='cluster')
      .join(rest_counts, on='cluster')
      .join(pub_counts,  on='cluster')
      .join(schl_counts, on='cluster')
      .fillna(0)
  )
```

```
# ============================================================
  # STEP 6: MAP PARTICIPANTS TO THEIR HOME CLUSTER
  # Use checkins: each participant's home = most visited Apartment
  # ============================================================
  print("Mapping participants to home clusters...")
```

```
apt_checkins = checkins[checkins['venueType'] == 'Apartment'].copy()
```

```
home_apt = (apt_checkins
      .groupby(['participantId', 'venueId'])
      .size()
      .reset_index(name='visits')
      .sort_values('visits', ascending=False)
      .drop_duplicates('participantId')
      .rename(columns={'venueId': 'apartmentId'})
  )
```

```
apt_to_cluster = gdf_apts.set_index('apartmentId')['cluster'].to_dict()
  home_apt['cluster'] = home_apt['apartmentId'].map(apt_to_cluster)
```

```
participants_merged = participants.merge(
      home_apt[['participantId', 'cluster']],
      on='participantId', how='left'
  )
```

```
matched = participants_merged['cluster'].notna().sum()
  print(f"  Participants matched to a cluster: {matched} / {len(participants)}")
```

```
# Per-cluster demographic stats
  part_by_cluster = participants_merged.groupby('cluster').agg(
      avg_joviality  = ('joviality',  'mean'),
      pct_kids       = ('haveKids',   'mean'),
      avg_age        = ('age',        'mean'),
      avg_household  = ('householdSize', 'mean'),
  ).reset_index()
```

```
cluster_summary = cluster_summary.merge(part_by_cluster, on='cluster', how='left')
```

```
# ============================================================
  # STEP 7: SMART ZONE CLASSIFICATION
  # Uses rent level + employer density + amenities + joviality
  # ============================================================
  global_avg_rent = gdf_apts['rentalCost'].mean()
  print(f"  City-wide average rent: ${global_avg_rent:.0f}")
```

```
def classify_zone(row):
      rent          = row['avg_rent']
      employers     = row['num_employers']
      restaurants   = row['num_restaurants']
      pubs          = row['num_pubs']
      schools       = row['num_schools']
      apts          = row['num_apartments']
      joviality     = row.get('avg_joviality', 0.5)
      com_density   = employers / apts if apts > 0 else 0
      amenity_score = restaurants + pubs
```

```
    if schools >= 1 and com_density < 0.1:
          return 'Educational / Family Zone'
      if com_density > 0.25:
          return 'Commercial / Employment Hub'
      if amenity_score >= 4 and rent >= global_avg_rent:
          return 'Affluent Entertainment District'
      if rent > global_avg_rent * 1.2:
          return 'Upscale Residential'
      if rent < global_avg_rent * 0.8:
          return 'Low-Income Residential'
      if joviality > 0.5 and amenity_score >= 2:
          return 'Mixed Residential (Active)'
      return 'Suburban Residential'
```

```
cluster_summary['zone_type'] = cluster_summary.apply(classify_zone, axis=1)
  gdf_apts = gdf_apts.merge(
      cluster_summary[['cluster', 'zone_type']], on='cluster', how='left'
  )
```

```
# ============================================================
  # STEP 8: PRINT FULL ZONE SUMMARY
  # ============================================================
  print("\n" + "="*70)
  print("ZONE SUMMARY — CITY OF ENGAGEMENT")
  print("="*70)
  for _, row in cluster_summary.sort_values('avg_rent', ascending=False).iterrows():
      print(f"\nZone {int(row['cluster'])} — {row['zone_type']}")
      print(f"  Apartments  : {int(row['num_apartments'])}")
      print(f"  Avg Rent    : ${row['avg_rent']:.0f}  "
            f"(range: ${row['min_rent']:.0f} - ${row['max_rent']:.0f})")
      print(f"  Avg Rooms   : {row['avg_rooms']:.1f}")
      print(f"  Employers   : {int(row['num_employers'])}  |  "
            f"Restaurants: {int(row['num_restaurants'])}  |  "
            f"Pubs: {int(row['num_pubs'])}  |  "
            f"Schools: {int(row['num_schools'])}")
      print(f"  Avg Age     : {row['avg_age']:.1f}  |  "
            f"Joviality: {row['avg_joviality']:.2f}  |  "
            f"Has Kids: {row['pct_kids']*100:.0f}%  |  "
            f"Household: {row['avg_household']:.1f}")
      print(f"  Center      : ({row['cx']:.0f}, {row['cy']:.0f})")
```

```
# ============================================================
  # COLOUR PALETTE FOR ZONES
  # ============================================================
  zone_palette = {
      'Educational / Family Zone':       '#2E7D32',
      'Commercial / Employment Hub':     '#E65100',
      'Affluent Entertainment District': '#7B1FA2',
      'Upscale Residential':             '#1565C0',
      'Low-Income Residential':          '#F44336',
      'Mixed Residential (Active)':      '#00ACC1',
      'Suburban Residential':            '#78909C',
  }
```

```
def zone_color(zone_name):
      return zone_palette.get(zone_name, '#9E9E9E')
```

```
# ============================================================
  # VIZ 1: City map — all venue types
  # ============================================================
  print("\nGenerating visualizations...")
```

```
fig, ax = plt.subplots(figsize=(14, 10))
  ax.set_facecolor('#f0f0f0')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white',
                     linewidth=0.2, alpha=0.4)
```

```
ax.scatter(gdf_apts.geometry.x, gdf_apts.geometry.y,
             c='#2196F3', s=8, alpha=0.5, label='Apartment (Residential)', zorder=3)
  ax.scatter(gdf_emp.geometry.x, gdf_emp.geometry.y,
             c='#FF9800', s=15, alpha=0.8, label='Employer (Commercial)', zorder=4)
  ax.scatter(gdf_rest.geometry.x, gdf_rest.geometry.y,
             c='red', marker='o', s=80, label='Restaurant', zorder=6)
  ax.scatter(gdf_pubs.geometry.x, gdf_pubs.geometry.y,
             c='purple', marker='s', s=70, label='Pub', zorder=6)
  ax.scatter(gdf_schl.geometry.x, gdf_schl.geometry.y,
             c='green', marker='^', s=130, label='School', zorder=6)
```

```
ax.legend(loc='lower right', fontsize=9)
  ax.set_title("City of Engagement — All Venue Types",
               fontsize=14, fontweight='bold')
  ax.set_xlabel("X Coordinate")
  ax.set_ylabel("Y Coordinate")
  plt.tight_layout()
  plt.savefig("q1_city_map.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("  Saved: q1_city_map.png")
```

```
# ============================================================
  # VIZ 2: Identified zones map with labels
  # ============================================================
  fig, ax = plt.subplots(figsize=(14, 10))
  ax.set_facecolor('#e8e8e8')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white',
                     linewidth=0.2, alpha=0.4)
```

```
for zone_name, color in zone_palette.items():
      subset = gdf_apts[gdf_apts['zone_type'] == zone_name]
      if len(subset):
          ax.scatter(subset.geometry.x, subset.geometry.y,
                     c=color, s=12, alpha=0.75,
                     label=zone_name, zorder=4)
```

```
# Label each zone centroid
  for _, row in cluster_summary.iterrows():
      ax.annotate(
          f"Zone {int(row['cluster'])}\n{row['zone_type']}\n"
          f"Avg Rent: ${row['avg_rent']:.0f}",
          xy=(row['cx'], row['cy']),
          fontsize=6.5, ha='center',
          bbox=dict(boxstyle='round,pad=0.3', fc='white',
                    alpha=0.92, ec='grey', linewidth=0.8)
      )
```

```
ax.legend(loc='lower right', fontsize=8, title="Zone Type")
  ax.set_title("City of Engagement — Identified Zones (K-Means, k=6)",
               fontsize=14, fontweight='bold')
  ax.set_xlabel("X Coordinate")
  ax.set_ylabel("Y Coordinate")
  plt.tight_layout()
  plt.savefig("q1_zones_map.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("  Saved: q1_zones_map.png")
```

```
# ============================================================
  # VIZ 3: Zone profiles — rent, amenities, happiness
  # ============================================================
  cs = cluster_summary.sort_values('avg_rent', ascending=True)
  labels  = [f"Z{int(r['cluster'])}: {r['zone_type']}"
             for _, r in cs.iterrows()]
  colors  = [zone_color(z) for z in cs['zone_type']]
  y_pos   = np.arange(len(labels))
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

```
# Panel 1 — Average rent
  axes[0].barh(y_pos, cs['avg_rent'], color=colors, edgecolor='white', height=0.6)
  axes[0].set_yticks(y_pos)
  axes[0].set_yticklabels(labels, fontsize=8)
  axes[0].set_xlabel("Avg Monthly Rent ($)")
  axes[0].set_title("Average Rental Cost", fontweight='bold')
  axes[0].axvline(global_avg_rent, color='black', linestyle='--',
                  linewidth=1.2, label=f"City avg ${global_avg_rent:.0f}")
  axes[0].legend(fontsize=8)
```

```
# Panel 2 — Amenity access (restaurants + pubs)
  amenity = cs['num_restaurants'] + cs['num_pubs']
  axes[1].barh(y_pos, amenity, color=colors, edgecolor='white', height=0.6)
  axes[1].set_yticks(y_pos)
  axes[1].set_yticklabels(labels, fontsize=8)
  axes[1].set_xlabel("Restaurants + Pubs nearby")
  axes[1].set_title("Amenity Access", fontweight='bold')
```

```
# Panel 3 — Resident joviality
  axes[2].barh(y_pos, cs['avg_joviality'], color=colors, edgecolor='white', height=0.6)
  axes[2].set_yticks(y_pos)
  axes[2].set_yticklabels(labels, fontsize=8)
  axes[2].set_xlabel("Average Joviality (0-1)")
  axes[2].set_title("Resident Happiness", fontweight='bold')
  axes[2].axvline(0.5, color='black', linestyle='--', linewidth=1.2)
```

```
plt.suptitle("Zone Profiles — Rent, Amenity Access & Resident Happiness",
               fontsize=13, fontweight='bold')
  plt.tight_layout()
  plt.savefig("q1_zone_composition.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("  Saved: q1_zone_composition.png")
```

```
# ============================================================
  # VIZ 4: Rental cost heatmap (wealthy vs low-income)
  # ============================================================
  fig, ax = plt.subplots(figsize=(13, 9))
  ax.set_facecolor('#f0f0f0')
  gdf_buildings.plot(ax=ax, color='#ddd', edgecolor='white',
                     linewidth=0.2, alpha=0.4)
```

```
sc = ax.scatter(
      gdf_apts.geometry.x, gdf_apts.geometry.y,
      c=gdf_apts['rentalCost'],
      cmap='RdYlGn_r',
      s=20, alpha=0.8, zorder=5
  )
  cbar = plt.colorbar(sc, ax=ax, shrink=0.6)
  cbar.set_label("Monthly Rent ($)", fontsize=10)
  ax.set_title("Apartment Rental Cost — Wealthy vs Low-Income Areas",
               fontsize=13, fontweight='bold')
  ax.set_xlabel("X Coordinate")
  ax.set_ylabel("Y Coordinate")
  plt.tight_layout()
  plt.savefig("q1_rental_heatmap.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("  Saved: q1_rental_heatmap.png")
```

```
# ============================================================
  # VIZ 5: Checkin density by activity type (4 subplots)
  # ============================================================
  venue_locations = []
  id_cols = {
      'Apartment': ('apartmentId', gdf_apts),
      'Workplace':  ('employerId',  gdf_emp),
      'Restaurant': ('restaurantId',gdf_rest),
      'Pub':        ('pubId',       gdf_pubs),
  }
```

```
for vtype, (id_col, gdf) in id_cols.items():
      subset = checkins[checkins['venueType'] == vtype].copy()
      lookup = gdf.set_index(id_col)[['geometry']].copy()
      for _, row in subset.iterrows():
          vid = row['venueId']
          if vid in lookup.index:
              g = lookup.loc[vid, 'geometry']
              venue_locations.append({
                  'venueType': vtype,
                  'x': g.x, 'y': g.y
              })
```

```
loc_df = pd.DataFrame(venue_locations)
```

```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
  configs = [
      ('Apartment',  'Home Activity',       '#1976D2'),
      ('Workplace',  'Workplace Activity',  '#F57C00'),
      ('Restaurant', 'Restaurant Activity', '#D32F2F'),
      ('Pub',        'Pub / Social Activity','#7B1FA2'),
  ]
```

```
for ax, (vt, title, col) in zip(axes.flat, configs):
      ax.set_facecolor('#f5f5f5')
      gdf_buildings.plot(ax=ax, color='#e0e0e0', edgecolor='white',
                         linewidth=0.2, alpha=0.5)
      sub = loc_df[loc_df['venueType'] == vt]
      if len(sub):
          ax.scatter(sub['x'], sub['y'], c=col, s=5, alpha=0.3)
      ax.set_title(title, fontweight='bold', fontsize=11)
      ax.set_xlabel("X")
      ax.set_ylabel("Y")
```

```
plt.suptitle("Checkin Density by Activity Type\n"
             "(reveals functional specialization of city areas)",
             fontsize=14, fontweight='bold')
  plt.tight_layout()
  plt.savefig("q1_checkin_density.png", dpi=150, bbox_inches='tight')
  plt.show()
  print("  Saved: q1_checkin_density.png")
```

```
print("\n✅ All 5 visualizations saved. Analysis complete.")
```

What this complete script does, end to end

Step	What happens
1	Loads all 8 CSVs, strips hidden spaces from all column names
2	Parses WKT geometry strings into Shapely Point/Polygon objects
3	Runs K-Means (k=6) **on apartments only** — best proxy for neighbourhoods
4	Assigns employers/restaurants/pubs/schools to nearest cluster via KD-Tree
5	Builds a profile per cluster: rent stats, room count, venue counts
6	Maps each participant to their home cluster via most-visited Apartment in checkins
7	Classifies each zone using rent + employer density + amenities + joviality
8	Prints a full readable summary to the console
Viz 1	City overview — all venue types as coloured points
Viz 2	Zones map — apartments coloured by zone type with centroid labels
Viz 3	3-panel horizontal bar chart: rent / amenities / happiness per zone
Viz 4	Rental cost heatmap — red=expensive, green=cheap
Viz 5	4-panel checkin density — home / work / restaurant / pub activity

Here is the complete optimized script from start to finish:

Q10: User

Challenge 2: Patterns of Life considers the patterns of daily life throughout the city. You will describe the daily routines for some representative people, characterize the travel patterns to identify potential bottlenecks or hazards, and examine how these patterns change over time and seasons. In Challenge 2, you will use visual analytic techniques to address these questions:

- Assuming the volunteers are representative of the city's population, characterize the distinct areas of the city that you identify. For each area you identify, provide your rationale and supporting data. Limit your response to 10 images and 500 words.
- Where are the busiest areas in Engagement? Are there traffic bottlenecks that should be addressed? Explain your rationale. Limit your response to 10 images and 500 words.
- Participants have given permission to have their daily routines captured. Choose two different participants with different routines and describe their daily patterns, with supporting evidence. Limit your response to 10 images and 500 words.
- Over the span of the dataset, how do patterns change in the city change? Describe up to 10 significant changes, with supporting evidence. Limit your response to 10 images and 500 words

Research and get solutions from the internet for this challenge. Provide the complete solutions for this entire challenge. Provide for each subpoint one interface implemnted in css/html/jsx/javascript/d3 and other things which are necessary for the implementation. It should look like something similiar in the visual studio code. Provide all steps in detail and if is necessary to run commands on the terminal.

A10: Claude

Check frontend design skill

Check frontend design skill

This is a large implementation. Let me write every file completely, in order, so you can build the full app.

💻 Step 1 — Terminal: Create the project

bash

```
# In VS Code terminal, navigate to where you want the project
  cd "C:\Users\Joao Castro\Documents\Uni\Master\Semester 2\High Performance Data Analyt
  ics and Visualization\Assignments"
```

```
npm create vite@latest HPCDA-VAST-Challenge2 -- --template react
  cd HPCDA-VAST-Challenge2
  npm install d3 @reduxjs/toolkit react-redux papaparse
```

```
# Create all folders
  mkdir src\components\Q1_CityAreas
  mkdir src\components\Q2_Traffic
  mkdir src\components\Q3_Routines
  mkdir src\components\Q4_Patterns
  mkdir src\redux
  mkdir src\utils
  mkdir public\data
```

🐍 Step 2 — Run Python preprocessing first

Save this as `preprocess.py` in your Downloads folder and run it:

Python preprocessing script for VAST Challenge data

Python preprocessing script for VAST Challenge data

Now every source file for the app:

Created 11 files

Created 11 files

Now the four question components:

Created 7 files

Created 7 files

The implementation is very large. Here's exactly what to do to get everything working:

📂 Complete File List Created

The files created in `/mnt/user-data/outputs/` are all the ones you need. Let me finalize the remaining 3 component files inline here:

`src/components/Q3_Routines/RoutinesContainer.jsx`:

jsx

```
import './Routines.css';
  import { useEffect, useRef, useMemo } from 'react';
  import { useSelector, useDispatch } from 'react-redux';
  import { setParticipant1, setParticipant2 } from '../../redux/SelectionSlice';
  import RoutinesD3 from './Routines-d3';
  import { COLORS } from '../../utils/geoUtils';
```

```
const MODE_LEGEND = [
    { color: COLORS.AtHome,       label: 'Home (Apartment)' },
    { color: COLORS.AtWork,       label: 'Work (Employer)'  },
    { color: COLORS.AtRestaurant, label: 'Restaurant'       },
    { color: COLORS.AtRecreation, label: 'Pub/Recreation'   },
  ];
```

```
export default function RoutinesContainer() {
    const dispatch = useDispatch();
    const { checkins, participants, status } = useSelector(s => s.vastData);
    const { participant1, participant2 }    = useSelector(s => s.selection);
```

```
  const panel1Ref = useRef(null);
    const panel2Ref = useRef(null);
    const d3Ref     = useRef(null);
    const wrapRef   = useRef(null);
```

```
  const pIds = useMemo(() =>
      [...new Set(checkins.map(c => c.participantId))].sort((a,b)=>a-b),
      [checkins]
    );
```

```
const getCheckins = (pid) =>
    checkins.filter(c => c.participantId === pid && c.date);
  const getInfo = (pid) =>
    participants.find(p => p.participantId === pid);
```

```
useEffect(() => {
    const d3 = new RoutinesD3(wrapRef.current);
    d3.create({ size: { width: wrapRef.current.offsetWidth, height: wrapRef.current.o
ffsetHeight }});
    d3Ref.current = d3;
    return () => d3.clear();
  }, []);
```

```
useEffect(() => {
    if (status !== 'loaded' || !d3Ref.current) return;
    setTimeout(() => {
      d3Ref.current.renderParticipant(
        panel1Ref.current,
        getCheckins(participant1),
        getInfo(participant1),
        Participant ${participant1}
      );
      d3Ref.current.renderParticipant(
        panel2Ref.current,
        getCheckins(participant2),
        getInfo(participant2),
        Participant ${participant2}
      );
    }, 100);
  }, [status, participant1, participant2, checkins]);
```

```
return (
    <div className="panel-wrap">
      <aside className="panel-sidebar">
        <div className="sidebar-section">
          <div className="sidebar-title">Q3 — Daily Routines</div>
          <p className="sidebar-text">
            Each dot = one check-in. X-axis = hour of day. Y-axis = date.
            Hover dots for details.
          </p>
        </div>
        <div className="sidebar-section">
          <div className="sidebar-title">Select Participants</div>
          <label className="ctrl-label">Participant 1</label>
          <select className="ctrl-select" value={participant1}
            onChange={e => dispatch(setParticipant1(+e.target.value))}>
            {pIds.map(id => <option key={id} value={id}>{id}</option>)}
          </select>
          <label className="ctrl-label" style={{marginTop:'8px'}}>Participant 2</labe
l>
          <select className="ctrl-select" value={participant2}
            onChange={e => dispatch(setParticipant2(+e.target.value))}>
            {pIds.map(id => <option key={id} value={id}>{id}</option>)}
          </select>
        </div>
        <div className="sidebar-section">
          <div className="sidebar-title">Activity Legend</div>
          {MODE_LEGEND.map(l => (
            <div key={l.label} className="legend-item">
              <div className="legend-dot" style={{ background: l.color }} />
              <span>{l.label}</span>
            </div>
          ))}
        </div>
      </aside>
      <div ref={wrapRef} className="panel-main routines-main">
        <div ref={panel1Ref} className="routines-panel" />
        <div className="routines-divider" />
        <div ref={panel2Ref} className="routines-panel" />
        <div className="vast-tooltip" style={{visibility:'hidden',position:'fixed'}}
/>
      </div>
    </div>
```

```
    );
  }
```

`src/components/Q3_Routines/Routines.css`:

css

```
.routines-main { display: flex; flex-direction: row; }
  .routines-panel { flex: 1; min-width: 0; height: 100%; }
  .routines-divider { width: 1px; background: #1e2d4a; flex-shrink: 0; }
```

`src/components/Q4_Patterns/PatternsContainer.jsx`:

jsx

```
import './Patterns.css';
  import { useEffect, useRef, useMemo } from 'react';
  import { useSelector, useDispatch } from 'react-redux';
  import { setSelectedMetric } from '../../redux/SelectionSlice';
  import * as d3 from 'd3';
```

```
const METRICS = [
    { id: 'checkins',    label: 'Total Check-ins' },
    { id: 'Apartment',   label: 'Home Visits' },
    { id: 'Workplace',   label: 'Work Visits' },
    { id: 'Restaurant',  label: 'Restaurant Visits' },
    { id: 'Pub',         label: 'Pub Visits' },
  ];
```

```
export default function PatternsContainer() {
    const dispatch = useDispatch();
    const { monthlyStats, monthlyFinancial, status } = useSelector(s => s.vastData);
    const selectedMetric = useSelector(s => s.selection.selectedMetric);
    const chartRef = useRef(null);
```

```
const chartData = useMemo(() => {
    if (!monthlyStats) return [];
    return Object.entries(monthlyStats)
      .map(([month, stats]) => ({
        month,
        value: selectedMetric === 'checkins'
          ? stats.total
          : (stats.byType?.[selectedMetric] || 0),
      }))
      .sort((a,b) => a.month.localeCompare(b.month));
  }, [monthlyStats, selectedMetric]);
```

```
useEffect(() => {
    if (!chartData.length || !chartRef.current) return;
    const el = chartRef.current;
    d3.select(el).selectAll('*').remove();
```

```
  const margin = { top: 30, right: 30, bottom: 60, left: 70 };
    const W = el.offsetWidth  - margin.left - margin.right;
    const H = el.offsetHeight - margin.top  - margin.bottom;
```

```
  const svg = d3.select(el).append('svg')
      .attr('width', el.offsetWidth).attr('height', el.offsetHeight)
      .style('background', 'transparent');
    const g = svg.append('g').attr('transform', translate(${margin.left},${margin.top}));
```

```
  const xS = d3.scalePoint().domain(chartData.map(d=>d.month)).range([0,W]).padding
(0.2);
    const yS = d3.scaleLinear().domain([0, d3.max(chartData, d=>d.value)*1.1 || 1]).r
ange([H,0]);
```

```
  // Grid
    g.selectAll('.grid-y').data(yS.ticks(5)).join('line').attr('class','grid-y')
      .attr('x1',0).attr('x2',W).attr('y1',d=>yS(d)).attr('y2',d=>yS(d))
      .attr('stroke','#1e2d4a').attr('stroke-width',0.8);
```

```
  // Area
    const area = d3.area().x(d=>xS(d.month)).y0(H).y1(d=>yS(d.value)).curve(d3.curveC
atmullRom);
    g.append('path').datum(chartData).attr('d', area)
      .attr('fill','rgba(0,212,200,0.1)').attr('stroke','none');
```

```
  // Line
    const line = d3.line().x(d=>xS(d.month)).y(d=>yS(d.value)).curve(d3.curveCatmullR
om);
    g.append('path').datum(chartData).attr('d', line)
      .attr('fill','none').attr('stroke','#00d4c8').attr('stroke-width',2.5);
```

```
  // Dots
    const tt = d3.select(el).append('div').attr('class','vast-tooltip').style('visibi
lity','hidden').style('position','fixed');
    g.selectAll('.dot').data(chartData).join('circle').attr('class','dot')
      .attr('cx',d=>xS(d.month)).attr('cy',d=>yS(d.value)).attr('r',4)
      .attr('fill','#00d4c8').attr('stroke','#0b0f19').attr('stroke-width',2)
      .on('mouseover',(e,d)=>tt.style('visibility','visible')
        .html(<strong>${d.month}</strong><span>${d.value.toLocaleString()} ${selectedMetric}</span
>)
        .style('left',(e.clientX+14)+'px').style('top',(e.clientY-10)+'px'))
      .on('mousemove',e=>tt.style('left',(e.clientX+14)+'px').style('top',(e.clientY-
10)+'px'))
      .on('mouseleave',()=>tt.style('visibility','hidden'));
```

```
  // Axes
    g.append('g').attr('transform',translate(0,${H}))
      .call(d3.axisBottom(xS).tickFormat(d=>d.slice(2)))
      .selectAll('text').attr('fill','#64748b').attr('font-size',9)
      .attr('transform','rotate(-40)').attr('text-anchor','end');
    g.append('g').call(d3.axisLeft(yS).ticks(5))
      .selectAll('text').attr('fill','#64748b').attr('font-size',9);
```

```
  svg.append('text').attr('x',margin.left).attr('y',20)
      .attr('font-size',13).attr('font-weight',600).attr('fill','#00d4c8')
      .attr('font-family','Syne,sans-serif').text(METRICS.find(m=>m.id===selectedMetr
ic)?.label);
  }, [chartData, selectedMetric]);
```

```
  return (
      <div className="panel-wrap">
        <aside className="panel-sidebar">
          <div className="sidebar-section">
            <div className="sidebar-title">Q4 — Pattern Changes</div>
            <p className="sidebar-text">Monthly trends across the 15-month study perio
d.</p>
          </div>
          <div className="sidebar-section">
            <div className="sidebar-title">Select Metric</div>
            {METRICS.map(m => (
              <button key={m.id}
                className={`q4-metric-btn ${selectedMetric===m.id?'active':''}`}
                onClick={()=>dispatch(setSelectedMetric(m.id))}>
                {m.label}
              </button>
            ))}
          </div>
          <div className="sidebar-section">
            <div className="sidebar-title">Findings</div>
            <p className="sidebar-text">
              Look for seasonal spikes in restaurant/pub visits during summer months,
              and dips in workplace visits during holiday periods. These reveal
              the rhythm of life in Engagement.
            </p>
          </div>
        </aside>
        <div ref={chartRef} className="panel-main q4-chart-area" />
      </div>
    );
  }
```

`src/components/Q4_Patterns/Patterns.css`:

```
css
```

```
.q4-chart-area { background: #0b0f19; }
  .q4-metric-btn {
    width: 100%; padding: 7px 10px; text-align: left;
    background: #161d2f; border: 1px solid #1e2d4a; border-radius: 5px;
    color: #64748b; font-family: 'Syne',sans-serif; font-size: 11px;
    cursor: pointer; transition: all 0.15s; margin-bottom: 3px;
  }
  .q4-metric-btn:hover { border-color: #00d4c8; color: #00d4c8; }
  .q4-metric-btn.active { border-color: #00d4c8; color: #00d4c8; background: rgba(0,21
  2,200,0.1); }
```

▶️ Step 3 — Run everything

bash

```
# 1. Run preprocessing (from project root, after copying preprocess.py there)
  python preprocess.py
```

```
# 2. Copy the generated public/data/ folder into HPCDA-VAST-Challenge2/
```

```
# 3. Install and start
  cd HPCDA-VAST-Challenge2
  npm install
  npm start
```

The app opens at **http://localhost:5173** with all 4 questions on tabs at the top.

This is a large implementation. Let me write every file completely, in order, so you can build the full app.

Save this as `preprocess.py` in your Downloads folder and run it:

Now every source file for the app:

Now the four question components:

The implementation is very large. Here's exactly what to do to get everything working:

The files created in `/mnt/user-data/outputs/` are all the ones you need. Let me finalize the remaining 3 component files inline here:

`src/components/Q3_Routines/RoutinesContainer.jsx`:

`src/components/Q3_Routines/Routines.css`:

`src/components/Q4_Patterns/PatternsContainer.jsx`:

`src/components/Q4_Patterns/Patterns.css`:

The app opens at **http://localhost:5173** with all 4 questions on tabs at the top.

Q11: User

According to the challenge chosen by your group, submit a short report including the following elements of Munzner's nested model seen in the first week: (5 points) Analysis of the domain and tasks (Who? What?) (5 points) Data and Task Abstraction (data types, data structures, data operations/transformations) (5x2 points) Two visual designs, duly justified, mentioning explicitly the mapping between data variables and visual variables and the choice of visual metaphor, if applicable. All design choices must be justified. A bonus will be granted for additional visual designs beyond the mandatory ones. Your report will contain a statement about the use, or lack of use, of LLMs for this assignment. Any use of LLMs must be described in detail (which LLM, version, etc.). A full copy of the related LLM conversations must be appended to the report. Provide the entire report in detail according to the assigment an the choosen challenge

A11: Claude

Ran 5 commands, viewed 2 files, created a file

Ran 5 commands, viewed 2 files, created a file

The report is ready. Here's a summary of what's inside each section:

📄 Report Structure

Section 1 — Domain and Tasks (5 pts)

- **Who** : Urban planning experts at desktop workstations, proficient with data tools
  - **What** : Four challenge questions mapped to user tasks, with a table showing data sources for each
  - **Why** : Addresses pain points of volume (670MB+ files), multivariate complexity, temporal depth, and spatial grounding

- **Validation** : Grader evaluation as surrogate user test

Section 2 — Data & Task Abstraction (5 pts)

- Full table of all key attributes with their Munzner data types (Quantitative/Ordinal/Nominal)
  - Three data structures: flat tables, event logs, spatial networks
  - Seven preprocessing operations (K-Means, spatial aggregation, temporal aggregation, WKT parsing, etc.)
  - Abstract task verbs per question: Discover/Summarize/Compare (Q1), Locate/Rank (Q2), Browse/Compare (Q3), Detect change/Correlate (Q4)

Section 3 — Visual Designs (5×2 = 10 pts + bonus)

- **Design 1 (Q1)** : Geographic map with zone clusters — full table of data→visual variable mappings with justifications
  - **Design 2 (Q3)** : Temporal dot plot for two participants — justified against two rejected alternatives
  - **Bonus Design 3 (Q2)** : Traffic heatmap with √-scaled circles
  - **Bonus Design 4 (Q4)** : Monthly time-series area chart

Section 4 — LLM Statement

- Exact model (Claude Sonnet 4.6), interface, dates
  - Table of what was accepted/rejected/adapted
  - Parts done entirely without LLM assistance

The report is ready. Here's a summary of what's inside each section:
Section 1 — Domain and Tasks (5 pts)

Section 2 — Data & Task Abstraction (5 pts)

Section 3 — Visual Designs (5×2 = 10 pts + bonus)

Section 4 — LLM Statement

Q12: User

Provide all the codes for question 2 in detail and give all the steps to run it properely

A12: Claude

Let me first check exactly what files and code already exist from the previous work, then build everything completely.

Ran 2 commands

Ran 2 commands

Good. Now I'll build the complete Q2 solution — Python preprocessing + full Vite/React/D3 app.

Created 9 files, ran a command

Created 9 files, ran a command

Now the five visualization components:

Created 5 files

Created 5 files

Now let's zip everything up and verify:

Ran 3 commands

Ran 3 commands

Complete Step-by-Step Guide

📂 Project Structure

```
VAST-Q2/
  ├── preprocess_q2.py          ← Run this FIRST (Python)
  ├── package.json
  ├── vite.config.js
  ├── index.html
  ├── public/
  │   └── data/                 ← preprocess writes JSON files here
  └── src/
      ├── index.jsx / index.css
      ├── App.jsx / App.css
      ├── utils/
      │   └── cityScales.js     ← shared D3 scales + color palette
      └── components/Q2_Traffic/
          ├── HeatmapView.jsx   ← Tab 1: spatial traffic heatmap
          ├── RankingView.jsx   ← Tab 2: busiest venues bar chart + map
          ├── HourlyView.jsx    ← Tab 3: check-ins by hour (stacked area)
          ├── WeekdayView.jsx   ← Tab 4: weekday × hour calendar heatmap
          └── FlowView.jsx      ← Tab 5: origin-destination travel flows
```

▶️ Step 1 — Run Python preprocessing

bash

```
# Edit preprocess_q2.py line 23 if needed:
  # BASE = Path(r"C:\Users\Joao Castro\Downloads\VAST-Challenge-2022\Datasets")
```

```
# Navigate to the VAST-Q2 folder, then:
  cd VAST-Q2
  python preprocess_q2.py
```

This reads the raw CSVs and writes 7 JSON files to `public/data/`. It takes 1–4 minutes depending on your machine. When done you'll see:

```
✅ Preprocessing complete!
    buildings.json       (XX KB)
    venues.json          (XX KB)
    checkin_grid.json    (XX KB)
    venue_ranking.json   (XX KB)
    hourly_profile.json  (XX KB)
    weekday_heatmap.json (XX KB)
    travel_flows.json    (XX KB)
```

▶️ Step 2 — Install and launch the app

bash

```
npm install
  npm start
  # Opens at http://localhost:5174
```

💻 What Each Tab Shows

Tab	Chart Type	Key Encoding
Traffic Heatmap	City map + circles	Size+color = check-in density per grid cell
Venue Ranking	Horizontal bar chart + map markers	Bar length + circle area = visit count
Hourly Profile	Stacked area chart	X = hour 0–23, Y = count, color = venue type
Weekday × Hour	Calendar heatmap	7×24 grid, color intensity = activity level
Travel Flows	Flow lines on city map	Line width = trip count, color = journey purpose

Let me first check exactly what files and code already exist from the previous work, then build everything completely.

Good. Now I'll build the complete Q2 solution — Python preprocessing + full Vite/React/D3 app.

Now the five visualization components:

Now let's zip everything up and verify:

This reads the raw CSVs and writes 7 JSON files to `public/data/`. It takes 1–4 minutes depending on your machine. When done you'll see:

Q13: User

Describe very briefly the justifications for the visual designs that don't have any description

A13: Claude

Viewed a file, ran a command

Viewed a file, ran a command

The document has 5 images. Images 1 and 2 already have justifications. **Images 3, 4, and 5 have none.** Based on their alt-text descriptions, here are brief justifications for each:

Image 3 — Multicoloured city zone map

This map colors the entire city grid by zone classification (residential, commercial, school). Color hue is used for the nominal zone-type variable — the most effective channel for categorical distinctions (per Mackinlay's ranking). The grid structure reduces individual-apartment clutter and supports region-level comparison, making spatial patterns immediately visible without requiring interaction.

Image 4 — Coloured dot map (individual apartments)

Each apartment appears as a dot whose color encodes a quantitative attribute (e.g., rent or joviality). Position encodes geographic location — the highest-ranked quantitative channel — while hue+saturation encodes the secondary variable. This layered encoding lets users simultaneously read spatial distribution and attribute values without switching views, supporting the task of identifying where high-rent or low-joviality clusters concentrate.

Image 5 — Horizontal bar chart

The bar chart ranks zones or categories by a quantitative metric (e.g., average rent, apartment count, or check-in frequency). Bar length (position from a common baseline) is used to encode quantity — the most accurate visual channel for ranked comparison (Cleveland & McGill 1984). The horizontal orientation accommodates longer category labels without rotation. This chart complements the maps by providing an exact numerical comparison that spatial views alone cannot support.

The document has 5 images. Images 1 and 2 already have justifications. **Images 3, 4, and 5 have none.** Based on their alt-text descriptions, here are brief justifications for each:

Image 3 — Multicoloured city zone map

This map colors the entire city grid by zone classification (residential, commercial, school). Color hue is used for the nominal zone-type variable — the most effective channel for categorical distinctions (per Mackinlay's ranking). The grid structure reduces individual-apartment clutter and supports region-level comparison, making spatial patterns immediately visible without requiring interaction.

Image 4 — Coloured dot map (individual apartments)

Each apartment appears as a dot whose color encodes a quantitative attribute (e.g., rent or joviality). Position encodes geographic location — the highest-ranked quantitative channel — while hue+saturation encodes the secondary variable. This layered encoding lets users simultaneously read spatial distribution and attribute values without switching views, supporting the task of identifying where high-rent or low-joviality clusters concentrate.

Image 5 — Horizontal bar chart

The bar chart ranks zones or categories by a quantitative metric (e.g., average rent, apartment count, or check-in frequency). Bar length (position from a common baseline) is used to encode quantity — the most accurate visual channel for ranked comparison (Cleveland & McGill 1984). The horizontal orientation accommodates longer category labels without rotation. This chart complements the maps by providing an exact numerical comparison that spatial views alone cannot support.
